MÁSTER EN REVERSING, ANÁLISIS DE MALWARE Y BUG HUNTING

MÁSTER EN *ANÁLISIS DE MALWARE Y REVERSING*

María Sonia Salido Fernández

Módulo 1 - Tarea 1

Campus Internacional CIBERSEGURIDAD

ENIIT INNOVA IT BUSINESS SCHOOL

UCAM UNIVERSIDAD CATÓLICA DE MURCIA

- 1. Entendiendo qué pide la tarea
  - 1.1 Entender la cadena de cifrado
  - 1.2 Recuperar la contraseña del certificado
  - 1.3 Atacar las claves RSA de los destinatarios
  - 1.4 Descifrar el correo mail.msg
  - 1.5 Siguientes pasos
  - 1.6 La pista importante
- 2. Analizar los certificados de bundle.pem
  - 2.1 Extraemos los 5 módulos n
- 3. Analizamos mail.msg
  - 3.1 Identificamos el tipo de fichero
  - 3.2 Analizamos el fichero
  - 3.3 Extraemos el bloque base64
- 4. Buscar una debilidad RSA entre los cinco destinatarios
  - 4.1 Calculo de todos los mcd(ni,nj)
  - 4.2 Ataque broadcast de Hastad
    - Conclusiones
- 5. Obtener la contraseña de EvilHacker.pfx
  - 5.1 Descifrar el encryptedContent con AES-256-CBC
- 6. Abrir EvilHacker.pfx
- 7. Extraer la clave privada de EvilHacker.pfx
- 8. Analizar y Descifrar EncryptedFolder.xml
- 9. Sacar la clave privada RSA de evilhacker.pem
- 10. FILE 1
- 11. FILE 2
- 12. FILE 3
- 13. Las credenciales de Gazprombank
- 14. Conclusiones

1. Entendiendo qué pide la tarea

1.1 Entender la cadena de cifrado

- La carpeta con las credenciales bancarias está en `EncryptedFolder.xml`, pero no podemos abrirla directamente porque su clave de cifrado está protegida.
- Esa clave, a su vez, está cifrada con la clave pública de un certificado que está en `EvilHacker.pfx`.
- El problema es que, aunque tenemos ese `.pfx`, no conocemos la contraseña del certificado, así que no podemos usar su clave privada todavía.

1.2 Recuperar la contraseña del certificado

- El enunciado dice que hemos interceptado un correo cifrado `mail.msg` que contiene la contraseña de acceso al certificado.
- Ese correo fue enviado a cinco destinatarios, y sus certificados públicos están en `bundle.pem`.
- Por tanto, el objetivo intermedio es:
  - analizar `mail.msg`,

- ver cómo está cifrado el correo para esos cinco destinatarios,
- usar los certificados de `bundle.pem`,
- y explotar alguna debilidad de RSA insinuada por la pista: “No todos los primos son iguales”. Esa pista sugiere que el ataque va por una debilidad en la generación de claves RSA, probablemente relacionada con los primos usados en los módulos de algunos destinatarios.

1.3 Atacar las claves RSA de los destinatarios

Tenemos que revisar los certificados de `bundle.pem` y extraer, para cada destinatario:

- El módulo RSA n.
- El exponente público e.

Después debemos comprobar si entre esas claves hay alguna vulnerabilidad, por ejemplo:

- Módulos que comparten un factor primo,
- claves mal generadas,
- o primos débiles o repetidos.

La pista `No todos los primos son iguales` apunta muy fuerte a que dos módulos RSA comparten un primo. Si eso ocurre, podemos calcular el mcd de dos módulos: $\gcd(n1, n2)$.

Si el resultado no es 1, hemos encontrado un factor primo común, y entonces podremos:

- Factorizar ambos módulos,
- reconstruir las claves privadas de esos destinatarios,
- y descifrar el correo o al menos la clave de sesión con la que fue protegido

1.4 Descifrar el correo mail.msg

Una vez recuperada la clave privada de uno de los destinatarios vulnerables, podremos intentar abrir el correo cifrado y obtener la contraseña del certificado `EvilHacker.pfx`.

1.5 Siguientes pasos

- Abrir `EvilHacker.pfx`.
- Descifrar `EncryptedFolder.xml`.
- Sacar las credenciales de `Gazprombank`.

1.6 La pista importante

La frase: `lee atentamente la Sección 4.3.2.3. No todos los primos son iguales` casi seguro nos está indicando que el ataque no va por fuerza bruta ni por adivinar contraseñas, sino por:

- Comparar los módulos RSA de los destinatarios.
- Calcular MCD entre ellos.
- Encontrar si comparten un primo.

---

2. Analizar los certificados de bundle.pem

```
-----BEGIN CERTIFICATE-----
MIICoTCCAgqgAwIBAgIUZh485tzuW/eIWgvP+TYUM+wQ5pkwDQYJKoZIhvcNAQEL
BQAwgY0xCzAJBgNVBAYTAlJVMRAwDgYDVQQIDAdJcmt1dHNrMRYwFAYDVQQHDA1O
aXpobmV1ZGluc2t5MRUwEwYDVQQKDAxiZWx5eSBtZWR2ZWQxGTAXBgNVBAMMEEV2
Z2VuaXkgQm9nYWNoZXYxIjAgBgkqhkiG9w0BCQEWE0V2Z2VuaXlAcHJvZGlrb3Yu
cnUwHhcNMjMwOTI0MTcyOTU2WhcNMjMxMDA0MTcyOTU2WjCBjTELMAkGA1UEBhMC
UlUxEDAOBgNVBAgMB0lya3V0c2sxFjAUBgNVBAcMDU5pemhuZXVkaW5za3kxFTAT
BgNVBAoMDGJlbHl5IG1lZHZlZDEZMBcGA1UEAwwQRXZnZW5peSBCb2dhY2hldjEi
MCAGCSqGSIb3DQEJARYTRXZnZW5peUBwcm9kaWtvdi5ydTCBnTANBgkqhkiG9w0B
AQEFAAOBiwAwgYcCgYEA3qWA6hHx3Le79g8ojD+MdHZ/RHSA/xuX1bmiNfoVH1T5
TobArPS/98lUc98936fodgywhMDvcGvI6xs6T502sly205m6HUe77g47KnDFhplp
/oPBnStB4xbcbIu4G7o4oryeANPinkvEISk+oPXL6oWVVzxXMVRfV9Q3QuD4ZnMC
AQUwDQYJKoZIhvcNAQELBQADgYEAB5P26z+UR58JBZQ8h5PP9j3UpTEnbQLHU483
u813zXWx9dzBwErsBS5infle0Xb5mkzfjEOjJuV5p4G9aN9+scYKW5BJdrBxpZDJ
yTxZSCIUJneEwM0b+ZO92VGNkQ/G/f+kT2eY2SY8xYJyowPAP2gscf0Awx6WdFvD
RMFYDX0=
-----END CERTIFICATE-----
-----BEGIN CERTIFICATE-----
MIICnzCCAgigAwIBAgIUK66lLlFj6oJa2YIBKQ24YTwp2ccwDQYJKoZIhvcNAQEL
BQAwgYwxCzAJBgNVBAYTAlJVMRAwDgYDVQQIDAdJcmt1dHNrMRYwFAYDVQQHDA1O
aXpobmV1ZGluc2t5MRUwEwYDVQQKDAxiZWx5eSBtZWR2ZWQxFzAVBgNVBAMMDlZs
YWRpbWlyIExldmluMSMwIQYJKoZIhvcNAQkBFhRWbGFkaW1pckBwcm9kaWtvdi5y
dTAeFw0yMzA5MjQxNzI5NTdaFw0yMzEwMDQxNzI5NTdaMIGMMQswCQYDVQQGEwJS
VTEQMA4GA1UECAwHSXJrdXRzazEWMBQGA1UEBwwNTml6aG5ldWRpbnNreTEVMBMG
A1UECgwMYmVseXkgbWVkdmVkMRcwFQYDVQQDDA5WbGFkaW1pciBMZXZpbjEjMCEG
CSqGSIb3DQEJARYUVmxhZGltaXJAcHJvZGlrb3YucnUwgZ0wDQYJKoZIhvcNAQEB
BQADgYsAMIGHAoGBAKfSQmXbh78S05esyozqC0agYuQUuglm8BmqWY/N+ChwjWbn
qVpQpnQuc+T16QlEBrp5ewMcIo1bdUMQzmnwm9yt7lDNBTyNocMDIllDT7rTXSNp
lKsUid73VGNbuz5h9Kw+ayyaEJLry9NjRGwQxBbXJrBcy2SVyfNUB2TH1QQfAgEF
MA0GCSqGSIb3DQEBCwUAA4GBAICHtVclCIk7LdE2Zmwd/SXrvXuQcEsB0ZtE6RQp
cpP15jM5PkXxq56jzJk2LeO1Jd6yQLECAe1wn6zJwGhJquzOvkpuBw83PUbybEvO
Lr0fx01Tb0ftpNUfd6xolH1wFX+rWj9Ar+AZOUJSIzoQE6Q/cdP3BmxR1n4nRuwf
gK08
-----END CERTIFICATE-----
-----BEGIN CERTIFICATE-----
MIICmTCCAgKgAwIBAgIUKPu1yr7umxmf0b7vAwVx9BApm+8wDQYJKoZIhvcNAQEL
BQAwgYkxCzAJBgNVBAYTAlJVMRAwDgYDVQQIDAdJcmt1dHNrMRYwFAYDVQQHDA1O
aXpobmV1ZGluc2t5MRUwEwYDVQQKDAxiZWx5eSBtZWR2ZWQxFzAVBgNVBAMMDlBp
ZXRyIFN0ZXBhbm92MSAwHgYJKoZIhvcNAQkBFhFQaWV0ckBwcm9kaWtvdi5ydTAe
Fw0yMzA5MjQxNzI5NThaFw0yMzEwMDQxNzI5NThaMIGJMQswCQYDVQQGEwJSVTEQ
MA4GA1UECAwHSXJrdXRzazEWMBQGA1UEBwwNTml6aG5ldWRpbnNreTEVMBMGA1UE
CgwMYmVseXkgbWVkdmVkMRcwFQYDVQQDDA5QaWV0ciBTdGVwYW5vdjEgMB4GCSqG
SIb3DQEJARYRUGlldHJAcHJvZGlrb3YucnUwgZ0wDQYJKoZIhvcNAQEBBQADgYsA
MIGHAoGBAOULfRhHvbKODj+bOYhN621kqKDg6sPkongjpun2SHEi0a3ozJ9QswZj
b/1eHDevue4xs9K7UNo4UiNjAQOSKqIaCFbxEq1R6nMsysGLExXmrBnkfhAtAexk
utp6GmVbfeN5+7vLRYXLOMU0wEJXy+s3INHyUhYr/IKBg7tWFfzbAgEFMA0GCSqG
SIb3DQEBCwUAA4GBAHt9QpwodW3z8KXelr2f6i7+53t8EKpLMwpjYzidgM5Ota9r
nmsY2GRQo+kzNumnaT+yXgQ5HUj+gSTVKwMnqnf3k3HoFLqjizeLLacILXJLVJVd
NddN52OEPmJKQdHn32u8SDxlMkHe9dS5FpsHCZqVtkFEI39VoOXhHNUZlM0O
-----END CERTIFICATE-----
-----BEGIN CERTIFICATE-----
MIICnTCCAgagAwIBAgIUIfUppnLLQ7nUxXZgVRVJ/Ce9CZowDQYJKoZIhvcNAQEL
```

```
BQAwgYsxCzAJBgNVBAYTAlJVMRAwDgYDVQQIDAdJcmt1dHNrMRYwFAYDVQQHDA1O
aXpobmV1ZGluc2t5MRUwEwYDVQQKDAxiZWx5eSBtZWR2ZWQxGTAXBgNVBAMMEE11
cmF0IFVydGVtYmF5ZXYxIDAeBgkqhkiG9w0BCQEWEU11cmF0QHByb2Rpa292LnJ1
MB4XDTIzMDkyNDE3MzAwMFoXDTIzMTAwNDE3MzAwMFowgYsxCzAJBgNVBAYTAlJV
MRAwDgYDVQQIDAdJcmt1dHNrMRYwFAYDVQQHDA1OaXpobmV1ZGluc2t5MRUwEwYD
VQQKDAxiZWx5eSBtZWR2ZWQxGTAXBgNVBAMMEE11cmF0IFVydGVtYmF5ZXYxIDAe
BgkqhkiG9w0BCQEWEU11cmF0QHByb2Rpa292LnJ1MIGdMA0GCSqGSIb3DQEBAQUA
A4GLADCBhwKBgQCcCWSGWReoB5JIN6gV26J6n2JwBer9xAwJNkyZxm2O4NRisKD2
/+7begoHyNYu77sNX4qoYQsxniNjf2EnsIjbophj/k0nXYtcVNHATOWnuvnChdDG
+0fSs/XncF/Hz23fYu2ShClZlJAVGR3HWVqk7JrwK2/FUQL4SPAZDDo78wIBBTAN
BgkqhkiG9w0BAQsFAAOBgQCZuVMJyPNF2RuNfF6Ywmy1TD5nagy5zmsK0Lwpw40h
sz59kENmsPgafokhEBbzrBB34f2x+uOMLOQTCDwWJj2DUmMfDLIPXj3M1WV5weFo
coTG3eINMnkDZszxaCCQE3HCkgQ9NNYPIqSRVa3SPQppkTiT5/LUmw7pvoOxOfwr
eQ==
-----END CERTIFICATE-----
-----BEGIN CERTIFICATE-----
MIICrzCCAhigAwIBAgIUA54myIK5FJaP/W7wFdXqKtziI6gwDQYJKoZIhvcNAQEL
BQAwgZQxCzAJBgNVBAYTAlJVMRAwDgYDVQQIDAdJcmt1dHNrMRYwFAYDVQQHDA1O
aXpobmV1ZGluc2t5MRUwEwYDVQQKDAxiZWx5eSBtZWR2ZWQxHjAcBgNVBAMMFUdv
bnphbG9mZiBBbHZhcmV6b3NreTEkMCIGCSqGSIb3DQEJARYVR29uemFsb2ZmQHBy
b2Rpa292LnJ1MB4XDTIzMDkyNDE3MzAwMloXDTIzMTAwNDE3MzAwMlowgZQxCzAJ
BgNVBAYTAlJVMRAwDgYDVQQIDAdJcmt1dHNrMRYwFAYDVQQHDA1OaXpobmV1ZGlu
c2t5MRUwEwYDVQQKDAxiZWx5eSBtZWR2ZWQxHjAcBgNVBAMMFUdvbnphbG9mZiBB
bHZhcmV6b3NreTEkMCIGCSqGSIb3DQEJARYVR29uemFsb2ZmQHByb2Rpa292LnJ1
MIGdMA0GCSqGSIb3DQEBAQUAA4GLADCBhwKBgQDJyhnYiaj4aOXScEP1tQKgeOpu
QA+7DXnaA2oopWTqXjUdawTHf6qrK1wjZn9A7IzCTtrcBRmVs4YT3B7g1Hm7TpOh
2csi/ryjii+/+RZx4MFSVpeQzcbU3VRjDLFpdt7h1SD7kxYYP0OpvOaInkdCcFrr
x42/CUopbBu8hOYN0QIBBTANBgkqhkiG9w0BAQsFAAOBgQBKyIShl2IxebThikjl
+acLCtttUUFf9B54GH4+kIJSsIcenB1CkEBYc0lwguHyEHjJxbGg6obudrAaT82O
ADpxQSjWbJvJRjCfg1wWXq0uEGrFCuexVVW7MTquS+Lsx9+PtX64et5xvOamB6UN
g65Iym2D5nKtjBNS+gDz5TQkxQ==
-----END CERTIFICATE-----
```


Para analizar los certificados, tenemos que extraer de cada certificado RSA:

- El módulo n.
- El exponente público e.

Para ello, con OpenSSL, listamos los certificados del bundle y los examinamos:

```
└─$ openssl crl2pkcs7 -nocrl -certfile bundle.pem  | \
openssl pkcs7 -print_certs -text -noout
Certificate:
    Data:
        Version: 3 (0x2)
        Serial Number:
            66:1e:3c:e6:dc:ee:5b:f7:88:5a:0b:cf:f9:36:14:33:ec:10:e6:99
        Signature Algorithm: sha256WithRSAEncryption
        Issuer: C=RU, ST=Irkutsk, L=Nizhneudinsky, O=belyy medved,
CN=Evgeniy Bogachev/emailAddress=Evgeniy@prodikov.ru
        Validity
            Not Before: Sep 24 17:29:56 2023 GMT
```

```
            Not After : Oct  4 17:29:56 2023 GMT
        Subject: C=RU, ST=Irkutsk, L=Nizhneudinsky, O=belyy medved,
CN=Evgeniy Bogachev/emailAddress=Evgeniy@prodikov.ru
        Subject Public Key Info:
            Public Key Algorithm: rsaEncryption
                Public-Key: (1024 bit)
                Modulus:
                    00:de:a5:80:ea:11:f1:dc:b7:bb:f6:0f:28:8c:3f:
                    8c:74:76:7f:44:74:80:ff:1b:97:d5:b9:a2:35:fa:
                    15:1f:54:f9:4e:86:c0:ac:f4:bf:f7:c9:54:73:df:
                    3d:df:a7:e8:76:0c:b0:84:c0:ef:70:6b:c8:eb:1b:
                    3a:4f:9d:36:b2:5c:b6:d3:99:ba:1d:47:bb:ee:0e:
                    3b:2a:70:c5:86:99:69:fe:83:c1:9d:2b:41:e3:16:
                    dc:6c:8b:b8:1b:ba:38:a2:bc:9e:00:d3:e2:9e:4b:
                    c4:21:29:3e:a0:f5:cb:ea:85:95:57:3c:57:31:54:
                    5f:57:d4:37:42:e0:f8:66:73
                Exponent: 5 (0x5)
    Signature Algorithm: sha256WithRSAEncryption
    Signature Value:
        07:93:f6:eb:3f:94:47:9f:09:05:94:3c:87:93:cf:f6:3d:d4:
        a5:31:27:6d:02:c7:53:8f:37:bb:cd:77:cd:75:b1:f5:dc:c1:
        c0:4a:ec:05:2e:62:9d:f9:5e:d1:76:f9:9a:4c:df:8c:43:a3:
        26:e5:79:a7:81:bd:68:df:7e:b1:c6:0a:5b:90:49:76:b0:71:
        a5:90:c9:c9:3c:59:48:22:14:26:77:84:c0:cd:1b:f9:93:bd:
        d9:51:8d:91:0f:c6:fd:ff:a4:4f:67:98:d9:26:3c:c5:82:72:
        a3:03:c0:3f:68:2c:71:fd:00:c3:1e:96:74:5b:c3:44:c1:58:
        0d:7d

Certificate:
    Data:
        Version: 3 (0x2)
        Serial Number:
            2b:ae:a5:2e:51:63:ea:82:5a:d9:82:01:29:0d:b8:61:3c:29:d9:c7
        Signature Algorithm: sha256WithRSAEncryption
        Issuer: C=RU, ST=Irkutsk, L=Nizhneudinsky, O=belyy medved,
CN=Vladimir Levin/emailAddress=Vladimir@prodikov.ru
        Validity
            Not Before: Sep 24 17:29:57 2023 GMT
            Not After : Oct  4 17:29:57 2023 GMT
        Subject: C=RU, ST=Irkutsk, L=Nizhneudinsky, O=belyy medved,
CN=Vladimir Levin/emailAddress=Vladimir@prodikov.ru
        Subject Public Key Info:
            Public Key Algorithm: rsaEncryption
                Public-Key: (1024 bit)
                Modulus:
                    00:a7:d2:42:65:db:87:bf:12:d3:97:ac:ca:8c:ea:
                    0b:46:a0:62:e4:14:ba:09:66:f0:19:aa:59:8f:cd:
                    f8:28:70:8d:66:e7:a9:5a:50:a6:74:2e:73:e4:f5:
                    e9:09:44:06:ba:79:7b:03:1c:22:8d:5b:75:43:10:
                    ce:69:f0:9b:dc:ad:ee:50:cd:05:3c:8d:a1:c3:03:
                    22:59:43:4f:ba:d3:5d:23:69:94:ab:14:89:de:f7:
                    54:63:5b:bb:3e:61:f4:ac:3e:6b:2c:9a:10:92:eb:
                    cb:d3:63:44:6c:10:c4:16:d7:26:b0:5c:cb:64:95:
                    c9:f3:54:07:64:c7:d5:04:1f
```

```
                Exponent: 5 (0x5)
    Signature Algorithm: sha256WithRSAEncryption
    Signature Value:
        80:87:b5:57:25:08:89:3b:2d:d1:36:66:6c:1d:fd:25:eb:bd:
        7b:90:70:4b:01:d1:9b:44:e9:14:29:72:93:f5:e6:33:39:3e:
        45:f1:ab:9e:a3:cc:99:36:2d:e3:b5:25:de:b2:40:b1:02:01:
        ed:70:9f:ac:c9:c0:68:49:aa:ec:ce:be:4a:6e:07:0f:37:3d:
        46:f2:6c:4b:ce:2e:bd:1f:c7:4d:53:6f:47:ed:a4:d5:1f:77:
        ac:68:94:7d:70:15:7f:ab:5a:3f:40:af:e0:19:39:42:52:23:
        3a:10:13:a4:3f:71:d3:f7:06:6c:51:d6:7e:27:46:ec:1f:80:
        ad:3c

Certificate:
    Data:
        Version: 3 (0x2)
        Serial Number:
            28:fb:b5:ca:be:ee:9b:19:9f:d1:be:ef:03:05:71:f4:10:29:9b:ef
        Signature Algorithm: sha256WithRSAEncryption
        Issuer: C=RU, ST=Irkutsk, L=Nizhneudinsky, O=belyy medved, CN=Pietr
Stepanov/emailAddress=Pietr@prodikov.ru
        Validity
            Not Before: Sep 24 17:29:58 2023 GMT
            Not After : Oct  4 17:29:58 2023 GMT
        Subject: C=RU, ST=Irkutsk, L=Nizhneudinsky, O=belyy medved,
CN=Pietr Stepanov/emailAddress=Pietr@prodikov.ru
        Subject Public Key Info:
            Public Key Algorithm: rsaEncryption
                Public-Key: (1024 bit)
                Modulus:
                    00:e5:0b:7d:18:47:bd:b2:8e:0e:3f:9b:39:88:4d:
                    eb:6d:64:a8:a0:e0:ea:c3:e4:a2:78:23:a6:e9:f6:
                    48:71:22:d1:ad:e8:cc:9f:50:b3:06:63:6f:fd:5e:
                    1c:37:af:b9:ee:31:b3:d2:bb:50:da:38:52:23:63:
                    01:03:92:2a:a2:1a:08:56:f1:12:ad:51:ea:73:2c:
                    ca:c1:8b:13:15:e6:ac:19:e4:7e:10:2d:01:ec:64:
                    ba:da:7a:1a:65:5b:7d:e3:79:fb:bb:cb:45:85:cb:
                    38:c5:34:c0:42:57:cb:eb:37:20:d1:f2:52:16:2b:
                    fc:82:81:83:bb:56:15:fc:db
                Exponent: 5 (0x5)
    Signature Algorithm: sha256WithRSAEncryption
    Signature Value:
        7b:7d:42:9c:28:75:6d:f3:f0:a5:de:96:bd:9f:ea:2e:fe:e7:
        7b:7c:10:aa:4b:33:0a:63:63:38:9d:80:ce:4e:b5:af:6b:9e:
        6b:18:d8:64:50:a3:e9:33:36:e9:a7:69:3f:b2:5e:04:39:1d:
        48:fe:81:24:d5:2b:03:27:aa:77:f7:93:71:e8:14:ba:a3:8b:
        37:8b:2d:a7:08:2d:72:4b:54:95:5d:35:d7:4d:e7:63:84:3e:
        62:4a:41:d1:e7:df:6b:bc:48:3c:65:32:41:de:f5:d4:b9:16:
        9b:07:09:9a:95:b6:41:44:23:7f:55:a0:e5:e1:1c:d5:19:94:
        cd:0e

Certificate:
    Data:
        Version: 3 (0x2)
        Serial Number:
```

```
            21:f5:29:a6:72:cb:43:b9:d4:c5:76:60:55:15:49:fc:27:bd:09:9a
        Signature Algorithm: sha256WithRSAEncryption
        Issuer: C=RU, ST=Irkutsk, L=Nizhneudinsky, O=belyy medved, CN=Murat
Urtembayev/emailAddress=Murat@prodikov.ru
        Validity
            Not Before: Sep 24 17:30:00 2023 GMT
            Not After : Oct  4 17:30:00 2023 GMT
        Subject: C=RU, ST=Irkutsk, L=Nizhneudinsky, O=belyy medved,
CN=Murat Urtembayev/emailAddress=Murat@prodikov.ru
        Subject Public Key Info:
            Public Key Algorithm: rsaEncryption
                Public-Key: (1024 bit)
                Modulus:
                    00:9c:09:64:86:59:17:a8:07:92:48:37:a8:15:db:
                    a2:7a:9f:62:70:05:ea:fd:c4:0c:09:36:4c:99:c6:
                    6d:8e:e0:d4:62:b0:a0:f6:ff:ee:db:7a:0a:07:c8:
                    d6:2e:ef:bb:0d:5f:8a:a8:61:0b:31:9e:23:63:7f:
                    61:27:b0:88:db:a2:98:63:fe:4d:27:5d:8b:5c:54:
                    d1:c0:4c:e5:a7:ba:f9:c2:85:d0:c6:fb:47:d2:b3:
                    f5:e7:70:5f:c7:cf:6d:df:62:ed:92:84:29:59:94:
                    90:15:19:1d:c7:59:5a:a4:ec:9a:f0:2b:6f:c5:51:
                    02:f8:48:f0:19:0c:3a:3b:f3
                Exponent: 5 (0x5)
    Signature Algorithm: sha256WithRSAEncryption
    Signature Value:
        99:b9:53:09:c8:f3:45:d9:1b:8d:7c:5e:98:c2:6c:b5:4c:3e:
        67:6a:0c:b9:ce:6b:0a:d0:bc:29:c3:8d:21:b3:3e:7d:90:43:
        66:b0:f8:1a:7e:89:21:10:16:f3:ac:10:77:e1:fd:b1:fa:e3:
        8c:2c:e4:13:08:3c:16:26:3d:83:52:63:1f:0c:b2:0f:5e:3d:
        cc:d5:65:79:c1:e1:68:72:84:c6:dd:e2:0d:32:79:03:66:cc:
        f1:68:20:90:13:71:c2:92:04:3d:34:d6:0f:22:a4:91:55:ad:
        d2:3d:0a:69:91:38:93:e7:f2:d4:9b:0e:e9:be:83:b1:39:fc:
        2b:79

Certificate:
    Data:
        Version: 3 (0x2)
        Serial Number:
            03:9e:26:c8:82:b9:14:96:8f:fd:6e:f0:15:d5:ea:2a:dc:e2:23:a8
        Signature Algorithm: sha256WithRSAEncryption
        Issuer: C=RU, ST=Irkutsk, L=Nizhneudinsky, O=belyy medved,
CN=Gonzaloff Alvarezosky/emailAddress=Gonzaloff@prodikov.ru
        Validity
            Not Before: Sep 24 17:30:02 2023 GMT
            Not After : Oct  4 17:30:02 2023 GMT
        Subject: C=RU, ST=Irkutsk, L=Nizhneudinsky, O=belyy medved,
CN=Gonzaloff Alvarezosky/emailAddress=Gonzaloff@prodikov.ru
        Subject Public Key Info:
            Public Key Algorithm: rsaEncryption
                Public-Key: (1024 bit)
                Modulus:
                    00:c9:ca:19:d8:89:a8:f8:68:e5:d2:70:43:f5:b5:
                    02:a0:78:ea:6e:40:0f:bb:0d:79:da:03:6a:28:a5:
                    64:ea:5e:35:1d:6b:04:c7:7f:aa:ab:2b:5c:23:66:
```

```
                    7f:40:ec:8c:c2:4e:da:dc:05:19:95:b3:86:13:dc:
                    1e:e0:d4:79:bb:4e:93:a1:d9:cb:22:fe:bc:a3:8a:
                    2f:bf:f9:16:71:e0:c1:52:56:97:90:cd:c6:d4:dd:
                    54:63:0c:b1:69:76:de:e1:d5:20:fb:93:16:18:3f:
                    43:a9:bc:e6:88:9e:47:42:70:5a:eb:c7:8d:bf:09:
                    4a:29:6c:1b:bc:84:e6:0d:d1
                Exponent: 5 (0x5)
    Signature Algorithm: sha256WithRSAEncryption
    Signature Value:
        4a:c8:84:a1:97:62:31:79:b4:e1:8a:48:e5:f9:a7:0b:0a:db:
        6d:51:41:5f:f4:1e:78:18:7e:3e:90:82:52:b0:87:1e:9c:1d:
        42:90:40:58:73:49:70:82:e1:f2:10:78:c9:c5:b1:a0:ea:86:
        ee:76:b0:1a:4f:cd:8e:00:3a:71:41:28:d6:6c:9b:c9:46:30:
        9f:83:5c:16:5e:ad:2e:10:6a:c5:0a:e7:b1:55:55:bb:31:3a:
        ae:4b:e2:ec:c7:df:8f:b5:7e:b8:7a:de:71:bc:e6:a6:07:a5:
        0d:83:ae:48:ca:6d:83:e6:72:ad:8c:13:52:fa:00:f3:e5:34:
        24:c5
```

Donde:

- Tenemos un certificado por destinatario del correo:
  - `Evgeniy Bogachev`.
  - `Vladimir Levin`.
  - `Pietr Stepanov`.
  - `Murat Urtembayev`.
  - `Gonzaloff Alvarezosky`.
- Pertenecen a una misma organización o campaña:
  - País (C): RU (Rusia).
  - Estado (ST): Irkutsk.
  - Localidad (L): Nizhneudinsky.
  - Organización (O): belyy medved (Oso Blanco).
  - Dominio de correo: @prodikov.ru
- Son certificados `X.509` con las siguientes características:
  - RSA de 1024 bits.
  - **Exponente público: *e*= 5.**
  - Certificados Auto-firmados (Self-Signed): En todos los casos, el Issuer (Emisor) es idéntico al Subject (Sujeto). Esto indica que no hay una Autoridad de Certificación (CA) externa confiable → Issuer = Subject → Cada individuo firmó su propio certificado.
  - Validez muy corta, de unos 10 días.
  - **Módulos RSA de cada certificado:** En RSA, cada clave pública tiene la forma: (n,e). Vemos como ejemplo el primer certificado n_1:

```
                Modulus:
            00:de:a5:80:ea:11:f1:dc:b7:bb:f6:0f:28:8c:3f:
            8c:74:76:7f:44:74:80:ff:1b:97:d5:b9:a2:35:fa:
```

```
                15:1f:54:f9:4e:86:c0:ac:f4:bf:f7:c9:54:73:df:
                3d:df:a7:e8:76:0c:b0:84:c0:ef:70:6b:c8:eb:1b:
                3a:4f:9d:36:b2:5c:b6:d3:99:ba:1d:47:bb:ee:0e:
                3b:2a:70:c5:86:99:69:fe:83:c1:9d:2b:41:e3:16:
                dc:6c:8b:b8:1b:ba:38:a2:bc:9e:00:d3:e2:9e:4b:
                c4:21:29:3e:a0:f5:cb:ea:85:95:57:3c:57:31:54:
                5f:57:d4:37:42:e0:f8:66:73
```

- `00`: Este `00` forma parte de la codificación `ASN.1/DER` para indicar que el entero es positivo.
- El módulo n_1 como valor hexadecimal es:

  `dea580ea11f1dcb7bbf60f288c3f8c74767f447480ff1b97d5b9a235fa151f54f94e86c0acf4bff7c95473df3ddfa7e8760cb084c0ef706bc8eb1b3a4f9d36b25cb6d399ba1d47bbee0e3b2a70c5869969fe83c19d2b41e316dc6c8bb81bba38a2bc9e00d3e29e4bc421293ea0f5cbea8595573c5731545f57d43742e0f86673`

Destacamos que existe una debilidad Criptográfica:

- Tamaño de Llave RSA: 1024 bits se considera inseguro y vulnerable a ataques de factorizacion. Lo mínimo recomendado hoy es 2048 bits.
- Exponente Público: 5 (0x5). Es un valor inusualmente pequeño. Aunque no es intrínsecamente "roto", es una configuración atípica que a veces se usa para acelerar operaciones de cifrado en hardware limitado.

Pasos siguientes: Según los apuntes, podríamos escoger una de estas dos rutas:

- Håstad / broadcast RSA.
- Factor común entre módulos.

2.1 Extraemos los 5 módulos n

- n_1 - Evgeniy@prodikov.ru:
  - Hexadecimal : `dea580ea11f1dcb7bbf60f288c3f8c74767f447480ff1b97d5b9a235fa151f54f94e86c0acf4bff7c95473df3ddfa7e8760cb084c0ef706bc8eb1b3a4f9d36b25cb6d399ba1d47bbee0e3b2a70c5869969fe83c19d2b41e316dc6c8bb81bba38a2bc9e00d3e29e4bc421293ea0f5cbea8595573c5731545f57d43742e0f86673`.
  - Decimal: `15634768810272645482388941647245376963002989785351216625468777748607506303142318367625289534743248250024194668901335885547048667316958693067811108591164911612038812221578154571664279193151818661085464624362809044421362589886190605651518550768525345880360982620499930023529870923149058909420224779731270896190 7`
- n_2 - Vladimir@prodikov.ru:

- Hexadecimal:
  `a7d24265db87bf12d397acca8cea0b46a062e414ba0966f019aa598fcdf828708d66e7a95a50a6742e73e4f5e9094406ba797b031c228d5b754310ce69f09bdcadee50cd053c8da1c3032259434fbad35d236994ab1489def754635bbb3e61f4ac3e6b2c9a1092ebcbd363446c10c416d726b05ccb6495c9f3540764c7d5041f`.

- Decimal:
  `117848142580767301479140610710350191400363802862119303486145436424386403651945506124604731078881678785302566535894067779696338606654375587912054236667905745167180779541914821605115675550265348648694791854824634994884784098347860611336863256025292645012883471253871196440185889342884709931056940458285831881759`.

- n_3 - Pietr@prodikov.ru:

  - Hexadecimal:
    `e50b7d1847bdb28e0e3f9b39884deb6d64a8a0e0eac3e4a27823a6e9f6487122d1ade8cc9f50b306636ffd5e1c37afb9ee31b3d2bb50da385223630103922aa21a0856f112ad51ea732ccac18b1315e6ac19e47e102d01ec64bada7a1a655b7de379fbbbcb4585cb38c534c04257cbeb3720d1f252162bfc828183bb5615fcdb`.

  - Decimal:
    `109572690095833540015177630428364938495950572655072149382179286384922852732906777196360212411251388705330226247771356213394179053422433827490965557757599881601396359035094633961986603390874697313171100402004246306710761908700692386929283300038933941891301836103671930246156416077878801122403583579122661211123`.

- n_4 - Murat@prodikov.ru:

  - Hexadecimal:
    `9c0964865917a807924837a815dba27a9f627005eafdc40c09364c99c66d8ee0d462b0a0f6ffeedb7a0a07c8d62eefbb0d5f8aa8610b319e23637f6127b088dba29863fe4d275d8b5c54d1c04ce5a7baf9c285d0c6fb47d2b3f5e7705fc7cf6ddf62ed92842959949015191dc7595aa4ec9af02b6fc55102f848f0190c3a3bf3`.

  - Decimal:
    `141701375513385166436096856851870377569757196618323301515474561851963944201553892806385634685979719434134626892867514566869672175318054795127061123136343151046388750768853585184965464447778660441245369717291925456813496616014316315880303824874971158040757712062942215019738814733157445385148149570670759906769`.

- n_5 - Gonzaloff@prodikov.ru:

  - Hexadecimal:
    `c9ca19d889a8f868e5d27043f5b502a078ea6e400fbb0d79da036a28a564ea5e351d6b04c77faaab2b5c23667f40ec8cc24edadc051995b38613dc1ee0d479bb4e93a1d9cb22fe`

`bca38a2fbff91671e0c152569790cdc6d4dd54630cb16976dee1d520fb9316183f43a9bce6889e4742705aebc78dbf094a296c1bbc84e60dd1`.

- Decimal: `160840782788886869832552217679031227306145298760248558604239324564642932511809556758723771493024027652356917672632695677849531127766622621206334981013367305231061396447089403731525465581041597768882342165534924271303668908582936231000186385983869381704613464423017749139940223866684432378537046831053915159771`.

3. Analizamos mail.msg

Para saber cuál es la ruta de solución correcta vamos a analizar `mail.msg`. Así podremos responder a las preguntas:

- Qué formato criptográfico contiene realmente.
- Si dentro hay un objeto `S/MIME/CMS/PKCS#7` con varios destinatarios.

Un fichero con extensión `.msg` suele ser un correo de Microsoft Outlook, almacenado en formato `OLE/Compound File`. No siempre contiene el cuerpo legible directamente. A veces guarda:

- El cuerpo del mensaje,
- adjuntos,
- cabeceras MIME,
- un bloque `S/MIME/PKCS#7` embebido.

3.1 Identificamos el tipo de fichero

```
└─$ file mail.msg
mail.msg: news or mail, ASCII text, with CRLF line terminators
```

Donde:

- `mail.msg` no parece un fichero `Outlook/OLE` , sino correo en texto plano ASCII con finales CRLF.
- Conclusión: probablemente el nombre `.msg` es un engaño, y en realidad lo que tenemos es un mensaje `RFC822/MIME` o un fichero de correo plano.

3.2 Analizamos el fichero

```
└─$ cat mail.msg
To: "Dimitri" <dimitri@foo.com>
From: "Natasha" <natasha@bar.com>
Subject: Medved tantsuyet v snegu
MIME-Version: 1.0
Content-Type: application/pkcs7-mime; smime-type=enveloped-data;
name=smime.p7m
Content-Transfer-Encoding: base64
```

```
Content-Disposition: attachment; filename=smime.p7m

MIIGqAYJKoZIhvcNAQcDoIIGmTCCBpUCAQAxggZAMIIBOAIBADCBojCBiTELMAkG
A1UEBhMCUlUxEDAOBgNVBAgMB0lya3V0c2sxFjAUBgNVBAcMDU5pemhuZXVkaW5z
a3kxFTATBgNVBAoMDGJlbHl5IG1lZHZlZDEXMBUGA1UEAwwOUGlldHIgU3RlcGFu
b3YxIDAeBgkqhkiG9w0BCQEWEVBpZXRyQHByb2Rpa292LnJ1AhQo+7XKvu6bGZ/R
vu8DBXH0ECmb7zALBgkqhkiG9w0BAQEEgYB43wam5qgoYw3w5R5n/A9PcJW74LA/
0XkJAcLRZ63QHghFKi5/XkHEEgwYsM3jojw4QHuvNFLCPOz20rY+1nepKTHkNfqf
Utd2LzESWu7+xzF9/G07cSQniCWlTvCc0VUqn8Iu5B13/fAf31lduAjlX9c+5iwR
+Dhoo5QkWSdnBzCCAToCAQAwgaQwgYsxCzAJBgNVBAYTAlJVMRAwDgYDVQQIDAdJ
cmt1dHNrMRYwFAYDVQQHDA1OaXpobmV1ZGluc2t5MRUwEwYDVQQKDAxiZWx5eSBt
ZWR2ZWQxGTAXBgNVBAMMEE11cmF0IFVydGVtYmF5ZXYxIDAeBgkqhkiG9w0BCQEW
EU11cmF0QHByb2Rpa292LnJ1AhQh9SmmcstDudTFdmBVFUn8J70JmjALBgkqhkiG
9w0BAQEEgYAN7n/qzCFuea8+WjMBRlWRLvims7lfmR5UQBF8BeQ6sjeGzT/Ilg1L
MzCKHoMFBVLQIWDeq3SV8jI6z4+L8OuRfX7ISMB8oT99oAx9cMer5T/1jzbQAPYp
VOAp5d7qK9N2MXz2fjHf3R/o2P3kIoJrsOZDDJZT1c4w/B+8Enuz7jCCATsCAQAw
gaUwgYwxCzAJBgNVBAYTAlJVMRAwDgYDVQQIDAdJcmt1dHNrMRYwFAYDVQQHDA1O
aXpobmV1ZGluc2t5MRUwEwYDVQQKDAxiZWx5eSBtZWR2ZWQxFzAVBgNVBAMMDlZs
YWRpbWlyIExldmluMSMwIQYJKoZIhvcNAQkBFhRWbGFkaW1pckBwcm9kaWtvdi5y
dQIUK66lLlFj6oJa2YIBKQ24YTwp2ccwCwYJKoZIhvcNAQEBBIGAYppzq5wumEVA
wOblH5tREQIYg2DxGJcD4UxiVq4zf+3JlYaycObKr4BqyDQmDjxqy80gu0vHqqza
dAoOMGOvvUdcberDcDA8bG/cx8wOTw7CAy0OrRqEp5VapuApndVwQ/awVtAHlvYP
MkmygkJHzKaNcKqPbgaTryOtjt2V1icwggE8AgEAMIGmMIGNMQswCQYDVQQGEwJS
VTEQMA4GA1UECAwHSXJrdXRzazEWMBQGA1UEBwwNTml6aG5ldWRpbnNreTEVMBMG
A1UECgwMYmVseXkgbWVkdmVkMRkwFwYDVQQDDBBFdmdlbml5IEJvZ2FjaGV2MSIw
IAYJKoZIhvcNAQkBFhNFdmdlbml5QHByb2Rpa292LnJ1AhRmHjzm3O5b94haC8/5
NhQz7BDmmTALBgkqhkiG9w0BAQEEgYCTsvI9ISrU/iuK8s8JWsz1HUCkniKKRtsS
TuSJUVYHVbsJR+SXCf18MjchucdaGkILqv1FThPEyXKnqmpvMJJOaIzptmGxBObP
KwbNOO79118dr6e/N+QdHO2WsdanVs2N0V4bSBJm6+xasihWKCXhgVE/NdEIPXds
yAX3SKKA3jCCAUMCAQAwga0wgZQxCzAJBgNVBAYTAlJVMRAwDgYDVQQIDAdJcmt1
dHNrMRYwFAYDVQQHDA1OaXpobmV1ZGluc2t5MRUwEwYDVQQKDAxiZWx5eSBtZWR2
ZWQxHjAcBgNVBAMMFUdvbnphbG9mZiBBbHZhcmV6b3NreTEkMCIGCSqGSIb3DQEJ
ARYVR29uemFsb2ZmQHByb2Rpa292LnJ1AhQDnibIgrkUlo/9bvAV1eoq3OIjqDAL
BgkqhkiG9w0BAQEEgYC2hpfWbdg1usCv0HDUe3wEoxeC//wksxLqw+4rud0wsJbO
c83P0ZRmiTh6XIpnTO/5isXT6WHcG8GmLT+pHbyJGjUKf0TyTqudqRp6rEGYoYvH
b0h8HcXkyZvzhDS+nFLNsouD5Yfh2IqZEmBXHQkj4sMN76SXq9dtO3ooFmp9gDBM
BgkqhkiG9w0BBwEwHQYJYIZIAWUDBAEqBBCxqI4i1VgwWcwqH6H7WW+7gCDw6Ahb
+QfJaXmrnGu6TP71e3jQ5zMAY78tVIAnEzn0gw==
```

Donde:

- Destinatario del correo: `To → dimitri@foo.com`.
- Receptor del correo: `From → natasha@bar.com`.
- Adunto del correo: `Subject → Medved`.
- `MIME-Version: 1.0`: Indica que el mensaje sigue el estándar `MIME` para correos electrónicos. Esto implica que el correo no es sólo texto plano, sino que puede incluir tipos de contenido especiales, como por ejemplo: `application/pkcs7-mime`.
- `Content-Type: application/pkcs7-mime; smime-type=enveloped-data; name=smime.p7m`: Indica que el cuerpo del mensaje contiene un objeto `S/MIME de tipo`

`PKCS#7/CMS`:

  - `application/pkcs7-mime`: El contenido es un contenedor criptográfico `PKCS#7/CMS`.
  - `smime-type=enveloped-data`: Ese contenedor es de tipo `EnvelopedData`, es decir, contiene datos cifrados.
  - `name=smime.p7m`: El objeto se presenta como un fichero llamado `smime.p7m`.

- `Content-Transfer-Encoding: base64`: Indica que el contenido del mensaje se ha codificado en base64 para poder transportarse como texto ASCII dentro del correo. No significa que base64 sea el cifrado. El cifrado real está dentro del objeto `S/MIME`. `base64` sólo es una codificación de transporte para representar datos binarios en texto.

- Línea en blanco: La línea en blanco en `mail.msg` marca la separación entre las cabeceras `MIME` del correo y el cuerpo del mensaje. Después de la línea en blanco empieza el contenido real del mensaje: El bloque `base64` del objeto `smime.p7m`.

3.3 Extraemos el bloque base64

Extraemos el bloque base64:

```
 tr -d '\r' < mail.msg | awk 'BEGIN{p=0} /^$/{p=1; next} p{print}' >
smime.b64
```

Donde:

- `tr -d '\r' < mail.msg`: Lee el fichero `mail.msg` y elimina los caracteres `\r`. Esto es necesario ya que los correos suelen usar finales de línea CRLF, como por ejemplo `\r\n` o `\n`. Si no eliminamoss `\r`, la línea vacía que separa cabeceras y cuerpo no parece vacío del todo para algunas herramientas, porque realmente contiene un `\r`.
- `| awk 'BEGIN{p=0} /^$/{p=1; next} p{print}'`: Esto procesa la salida anterior línea por línea:
  - `BEGIN{p=0}`: Inicializa una variable `p` a `0`.
    - `p=0` significa: todavía estamos en las cabeceras.
    - `p=1` significará: ya hemos llegado al cuerpo. Mientras estemos en las cabeceras, no imprime nada. Pero cuando encuentra la línea vacía, empieza a imprimir todo lo que viene después.
- `> smime.b64`: Redirige la salida final al fichero `smime.b64`. Ese fichero contendrá sólo el cuerpo del mensaje, que en nuestro caso es el bloque base64 del objeto `S/MIME` que queremos obtener.

Mostramos la primera parte del fichero generado `smime.b64`:

```
└─$ head smime.b64
MIIGqAYJKoZIhvcNAQcDoIIGmTCCBpUCAQAxggZAMIIBOAIBADCBojCBiTELMAkG
A1UEBhMCUlUxEDAOBgNVBAgMB0lya3V0c2sxFjAUBgNVBAcMDU5pemhuZXVkaW5z
a3kxFTATBgNVBAoMDGJlbHl5IG1lZHZlZDEXMBUGA1UEAwwOUGlldHIgU3RlcGFu
b3YxIDAeBgkqhkiG9w0BCQEWEVBpZXRyQHByb2Rpa292LnJ1AhQo+7XKvu6bGZ/R
vu8DBXH0ECmb7zALBgkqhkiG9w0BAQEEgYB43wam5qgoYw3w5R5n/A9PcJW74LA/
0XkJAcLRZ63QHghFKi5/XkHEEgwYsM3jojw4QHuvNFLCPOz20rY+1nepKTHkNfqf
```

```
Utd2LzESWu7+xzF9/G07cSQniCWlTvCc0VUqn8Iu5B13/fAf31lduAjlX9c+5iwR
+Dhoo5QkWSdnBzCCAToCAQAwgaQwgYsxCzAJBgNVBAYTAlJVMRAwDgYDVQQIDAdJ
cmt1dHNrMRYwFAYDVQQHDA1OaXpobmV1ZGluc2t5MRUwEwYDVQQKDAxiZWx5eSBt
ZWR2ZWQxGTAXBgNVBAMMEE11cmF0IFVydGVtYmF5ZXYxIDAeBgkqhkiG9w0BCQEW
```

Donde:

- Vemos el bloque base64 del objeto `S/MIME` que hemos querido separar.

Decodificamos el fichero `smime.b64`:

```
└─$ base64 -d smime.b64 > smime.der
```

Donde:

- Se guarda en el fichero `smime.der` el resultado de decodificar el fichero `smime.b64` desde base64 a binario.

Leamos el fichero `smime.der` como un objeto CMS en formato DER y se imprime la estructura decodificada de forma legible:

```
└─$ openssl cms -inform DER -in smime.der -cmsout -print
CMS_ContentInfo:
  contentType: pkcs7-envelopedData (1.2.840.113549.1.7.3)
  d.envelopedData:
    version: 0
    originatorInfo: <ABSENT>
    recipientInfos:
      d.ktri:
        version: 0
        d.issuerAndSerialNumber:
          issuer:           C=RU, ST=Irkutsk, L=Nizhneudinsky, O=belyy
medved, CN=Pietr Stepanov, emailAddress=Pietr@prodikov.ru
          serialNumber: 0x28FBB5CABEEE9B199FD1BEEF030571F410299BEF
        keyEncryptionAlgorithm:
          algorithm: rsaEncryption (1.2.840.113549.1.1.1)
          parameter: <ABSENT>
        encryptedKey:
          0000 - 78 df 06 a6 e6 a8 28 63-0d f0 e5 1e 67 fc 0f   x.....
(c....g..
          000f - 4f 70 95 bb e0 b0 3f d1-79 09 01 c2 d1 67 ad
Op....?.y....g.
          001e - d0 1e 08 45 2a 2e 7f 5e-41 c4 12 0c 18 b0 cd
...E*..^A......
          002d - e3 a2 3c 38 40 7b af 34-52 c2 3c ec f6 d2 b6   ..<8@{.4R.
<....
          003c - 3e d6 77 a9 29 31 e4 35-fa 9f 52 d7 76 2f 31
>.w.)1.5..R.v/1
          004b - 12 5a ee fe c7 31 7d fc-6d 3b 71 24 27 88 25
```

```
.Z...1}.m;q$'.%
          005a - a5 4e f0 9c d1 55 2a 9f-c2 2e e4 1d 77 fd f0
.N...U*.....w..
          0069 - 1f df 59 5d b8 08 e5 5f-d7 3e e6 2c 11 f8 38
..Y]..._.>.,..8
          0078 - 68 a3 94 24 59 27 67 07-                      h..$Y'g.

      d.ktri:
        version: 0
        d.issuerAndSerialNumber:
          issuer:           C=RU, ST=Irkutsk, L=Nizhneudinsky, O=belyy
medved, CN=Murat Urtembayev, emailAddress=Murat@prodikov.ru
          serialNumber: 0x21F529A672CB43B9D4C57660551549FC27BD099A
        keyEncryptionAlgorithm:
          algorithm: rsaEncryption (1.2.840.113549.1.1.1)
          parameter: <ABSENT>
        encryptedKey:
          0000 - 0d ee 7f ea cc 21 6e 79-af 3e 5a 33 01 46 55
.....!ny.>Z3.FU
          000f - 91 2e f8 a6 b3 b9 5f 99-1e 54 40 11 7c 05 e4
......_..T@.|..
          001e - 3a b2 37 86 cd 3f c8 96-0d 4b 33 30 8a 1e 83
:.7..?...K30...
          002d - 05 05 52 d0 21 60 de ab-74 95 f2 32 3a cf 8f
..R.!`..t..2:..
          003c - 8b f0 eb 91 7d 7e c8 48-c0 7c a1 3f 7d a0 0c
....}~.H.|.?}..
          004b - 7d 70 c7 ab e5 3f f5 8f-36 d0 00 f6 29 54 e0
}p...?..6...)T.
          005a - 29 e5 de ea 2b d3 76 31-7c f6 7e 31 df dd 1f
)...+.v1|.~1...
          0069 - e8 d8 fd e4 22 82 6b b0-e6 43 0c 96 53 d5 ce
...."..k..C..S..
          0078 - 30 fc 1f bc 12 7b b3 ee-                      0....{..

      d.ktri:
        version: 0
        d.issuerAndSerialNumber:
          issuer:           C=RU, ST=Irkutsk, L=Nizhneudinsky, O=belyy
medved, CN=Vladimir Levin, emailAddress=Vladimir@prodikov.ru
          serialNumber: 0x2BAEA52E5163EA825AD98201290DB8613C29D9C7
        keyEncryptionAlgorithm:
          algorithm: rsaEncryption (1.2.840.113549.1.1.1)
          parameter: <ABSENT>
        encryptedKey:
          0000 - 62 9a 73 ab 9c 2e 98 45-40 c0 e6 e5 1f 9b 51
b.s....E@.....Q
          000f - 11 02 18 83 60 f1 18 97-03 e1 4c 62 56 ae 33
....`.....LbV.3
          001e - 7f ed c9 95 86 b2 70 e6-ca af 80 6a c8 34 26
......p....j.4&
          002d - 0e 3c 6a cb cd 20 bb 4b-c7 aa ac da 74 0a 0e   .<j..
.K....t..
          003c - 30 63 af bd 47 5c 6d ea-c3 70 30 3c 6c 6f dc
```

```
0c..G\m..p0<lo.
          004b - c7 cc 0e 4f 0e c2 03 2d-0e ad 1a 84 a7 95 5a
...O...-......Z
          005a - a6 e0 29 9d d5 70 43 f6-b0 56 d0 07 96 f6 0f
..)..pC..V.....
          0069 - 32 49 b2 82 42 47 cc a6-8d 70 aa 8f 6e 06 93
2I..BG...p..n..
          0078 - af 23 ad 8e dd 95 d6 27-                      .#.....'

      d.ktri:
        version: 0
        d.issuerAndSerialNumber:
          issuer:           C=RU, ST=Irkutsk, L=Nizhneudinsky, O=belyy
medved, CN=Evgeniy Bogachev, emailAddress=Evgeniy@prodikov.ru
          serialNumber: 0x661E3CE6DCEE5BF7885A0BCFF9361433EC10E699
        keyEncryptionAlgorithm:
          algorithm: rsaEncryption (1.2.840.113549.1.1.1)
          parameter: <ABSENT>
        encryptedKey:
          0000 - 93 b2 f2 3d 21 2a d4 fe-2b 8a f2 cf 09 5a cc
...=!*..+....Z.
          000f - f5 1d 40 a4 9e 22 8a 46-db 12 4e e4 89 51 56
..@..".F..N..QV
          001e - 07 55 bb 09 47 e4 97 09-fd 7c 32 37 21 b9 c7
.U..G....|27!..
          002d - 5a 1a 42 0b aa fd 45 4e-13 c4 c9 72 a7 aa 6a
Z.B...EN...r..j
          003c - 6f 30 92 4e 68 8c e9 b6-61 b1 04 e6 cf 2b 06
o0.Nh...a....+.
          004b - cd 38 ee fd d7 5f 1d af-a7 bf 37 e4 1d 1c ed
.8..._....7....
          005a - 96 b1 d6 a7 56 cd 8d d1-5e 1b 48 12 66 eb ec
....V...^.H.f..
          0069 - 5a b2 28 56 28 25 e1 81-51 3f 35 d1 08 3d 77   Z.(V(%..Q?
5..=w
          0078 - 6c c8 05 f7 48 a2 80 de-                      l...H...

      d.ktri:
        version: 0
        d.issuerAndSerialNumber:
          issuer:           C=RU, ST=Irkutsk, L=Nizhneudinsky, O=belyy
medved, CN=Gonzaloff Alvarezosky, emailAddress=Gonzaloff@prodikov.ru
          serialNumber: 0x039E26C882B914968FFD6EF015D5EA2ADCE223A8
        keyEncryptionAlgorithm:
          algorithm: rsaEncryption (1.2.840.113549.1.1.1)
          parameter: <ABSENT>
        encryptedKey:
          0000 - b6 86 97 d6 6d d8 35 ba-c0 af d0 70 d4 7b 7c
....m.5....p.{|
          000f - 04 a3 17 82 ff fc 24 b3-12 ea c3 ee 2b b9 dd
......$.....+..
          001e - 30 b0 96 ce 73 cd cf d1-94 66 89 38 7a 5c 8a
0...s....f.8z\.
          002d - 67 4c ef f9 8a c5 d3 e9-61 dc 1b c1 a6 2d 3f
```

```
gL......a....-?
          003c - a9 1d bc 89 1a 35 0a 7f-44 f2 4e ab 9d a9 1a
.....5..D.N....
          004b - 7a ac 41 98 a1 8b c7 6f-48 7c 1d c5 e4 c9 9b
z.A....oH|.....
          005a - f3 84 34 be 9c 52 cd b2-8b 83 e5 87 e1 d8 8a
..4..R.........
          0069 - 99 12 60 57 1d 09 23 e2-c3 0d ef a4 97 ab d7
..`W..#........
          0078 - 6d 3b 7a 28 16 6a 7d 80-                      m;z(.j}.
    encryptedContentInfo:
      contentType: pkcs7-data (1.2.840.113549.1.7.1)
      contentEncryptionAlgorithm:
        algorithm: aes-256-cbc (2.16.840.1.101.3.4.1.42)
        parameter: OCTET STRING:
          0000 - b1 a8 8e 22 d5 58 30 59-cc 2a 1f a1 fb 59 6f
...".X0Y.*...Yo
          000f - bb                                            .
      encryptedContent:
        0000 - f0 e8 08 5b f9 07 c9 69-79 ab 9c 6b ba 4c fe   ...
[...iy..k.L.
        000f - f5 7b 78 d0 e7 33 00 63-bf 2d 54 80 27 13 39   .{x..3.c.-
T.'.9
        001e - f4 83                                          ..
    unprotectedAttrs:
      <ABSENT>
```

Donde:

- Destinatario del correo que salía en la cabecera: `To: "Dimitri" <dimitri@foo.com>`.
- Destinatarios criptográficos del sobre `S/MIME`: Son los que aparecen en `recipientInfos`.
- `recipientInfos`: Aparecen cinco bloques `d.ktri:`. Cada bloque `d.ktri` es un `KeyTransRecipientInfo`, es decir, un destinatario criptográfico del `EnvelopedData`.
    - `issuer: C=RU, ST=Irkutsk, L=Nizhneudinsky, O=belyy medved, CN=Pietr Stepanov, emailAddress=Pietr@prodikov.ru`.
    - `issuer: C=RU, ST=Irkutsk, L=Nizhneudinsky, O=belyy medved, CN=Murat Urtembayev, emailAddress=Murat@prodikov.ru`.
    - `issuer: C=RU, ST=Irkutsk, L=Nizhneudinsky, O=belyy medved, CN=Vladimir Levin, emailAddress=Vladimir@prodikov.ru`.
    - `issuer: C=RU, ST=Irkutsk, L=Nizhneudinsky, O=belyy medved, CN=Evgeniy Bogachev, emailAddress=Evgeniy@prodikov.ru`.
    - `issuer: C=RU, ST=Irkutsk, L=Nizhneudinsky, O=belyy medved, CN=Gonzaloff Alvarezosky, emailAddress=Gonzaloff@prodikov.ru`.
- Cada bloque `d.ktri:`:
    - Tiene un destinatario criptográfico.
    - Está identificado por su `issuer + serial`.

    - Todos los destinatarios usan `rsaEncryption`.
    - Tiene una clave de sesión cifrada para él en `encryptedKey`. Estos `5 encryptedKey` son los `ciphertexts RSA c`.

- `IV - Vector de inicialización`: `b1 a8 8e 22 d5 58 30 59 cc 2a 1f a1 fb 59 6f bb`. Esto aparece en `parameter: OCTET STRING`:

```
algorithm: aes-256-cbc (2.16.840.1.101.3.4.1.42)
      parameter: OCTET STRING:
        0000 - b1 a8 8e 22 d5 58 30 59-cc 2a 1f a1 fb 59 6f .....
        000f - bb
```

Extraemos los 5 encryptedKey: OJO!!! Hay que ponerlos en orden para que coincidan con los valores de cada n_i. Aquí están cambiados el orden de Pietr y Murat!!!

- Para c_1 — Evgeniy Bogachev:

    - En hexadecimal: `93b2f23d212ad4fe2b8af2cf095accf51d40a49e228a46db124ee48951560755bb0947e49709fd7c323721b9c75a1a420baafd454e13c4c972a7aa6a6f30924e688ce9b661b104e6cf2b06cd38eefdd75f1dafa7bf37e41d1ced96b1d6a756cd8dd15e1b481266ebec5ab228562825e181513f35d1083d776cc805f748a280de`.

    - En decimal: `103717771130001008712491053180913848390031284537088910889032294491741283727856238500259232415716350939656923944123358269878028764570296929016776462393710532697965015421756333967982726200197069677063940096683594345498734503869482395109562783057380960212794916566657605442251083749951333173216813779355282079966`.

- Para c_2 — Vladimir Levin:

    - En hexadecimal: `629a73ab9c2e984540c0e6e51f9b511102188360f1189703e14c6256ae337fedc99586b270e6caaf806ac834260e3c6acbcd20bb4bc7aaacda740a0e3063afbd475c6deac370303c6c6fdcc7cc0e4f0ec2032d0ead1a84a7955aa6e0299dd57043f6b056d00796f60f3249b2824247cca68d70aa8f6e0693af23ad8edd95d627`.

    - En decimal: `692416112900216984497242428457247687600017599478589353975871156629870654948179020184611145744499749482908177004155336704751316597872907542245360024094429133422990488495145822696094253482121379477873398068908696911838547447029725226265049261065935291468845178755200464785266761882920089071253143224301307714 95`.

- Para c_3 — Murat Urtembayev:

- En hexadecimal:
  `0dee7feacc216e79af3e5a33014655912ef8a6b3b95f991e5440117c05e43ab23786cd3fc8960d4b33308a1e83050552d02160deab7495f2323acf8f8bf0eb917d7ec848c07ca13f7da00c7d70c7abe53ff58f36d000f62954e029e5deea2bd376317cf67e31dfdd1fe8d8fde422826bb0e6430c9653d5ce30fc1fbc127bb3ee`.

- En decimal:
  `97831298582238323153282064550053798277937577474473015002530764685898659169385375779874911366312861792060427098951263240922409631371908574344454579499128263473109126046595558332508827983521619647068285981795525610776040674993425186584648677568338940715853471660889437892194392314876026188288555482749869557 58`.

- Para c_4 — Pietr Stepanov:

  - En hexadecimal:
    `78df06a6e6a828630df0e51e67fc0f4f7095bbe0b03fd1790901c2d167add01e08452a2e7f5e41c4120c18b0cde3a23c38407baf3452c23cecf6d2b63ed677a92931e435fa9f52d7762f31125aeefec7317dfc6d3b7124278825a54ef09cd1552a9fc22ee41d77fdf01fdf595db808e55fd73ee62c11f83868a3942459276707`.

  - En decimal:
    `84878639806633832270987861815905523964403711759499006171717645388611199544731056680796026730128924861658270108230181325815544966035649270737605679830136247897230524638092114897633719946640362119791490448290465782482085518912343420398415592930979336927317304863095524830643610944119523986320824541223961716487`.

- Para c_5 — Gonzaloff Alvarezosky:

  - En hexadecimal:
    `b68697d66dd835bac0afd070d47b7c04a31782fffc24b312eac3ee2bb9dd30b096ce73cdcfd1946689387a5c8a674ceff98ac5d3e961dc1bc1a62d3fa91dbc891a350a7f44f24eab9da91a7aac4198a18bc76f487c1dc5e4c99bf38434be9c52cdb28b83e587e1d88a991260571d0923e2c30defa497abd76d3b7a28166a7d80`.

  - En decimal:
    `12817394357232556665396772174093188854194990816545538069788323425356757770019539144558680437271948537912298699874180531934010532050156627508209133387016843924675018877173606655017986566223208596958708707164613775067225291204718926210525679785699327867782303610264525352560282474069625173245656208831747997632 0`.

4. Buscar una debilidad RSA entre los cinco destinatarios

Sabemos:

- Del fichero `bundle.pem` sabemos los 5 módulos RSA `n` y el exponente público `e=5`.
- Del fichero `smime.der` sabemos los 5 `encryptedKey`, es decir, los `ciphertexts RSA c`.

Ahora podemos construir, para cada destinatario, su par: (n_i, c_i), donde:

- n_i = módulo RSA del certificado.
- c_i = `encryptedKey` del `RecipientInfo` correspondiente.

4.1 Calculo de todos los mcd(n_i,n_j)

Antes de desarrolar un ataque de broadcast, vamos a comprobar primero si hay módulos con factor común: gcd(n_i,n_j)

Script de Python para calcular directamente todos los gcd(n_i,n_j):

```
from math import gcd

n1 =
15634768810272645482388941647245376963002989785351216625468777486075063031
42318367625289534743248250024194668901335885547048667316958693067811108591 1
64911612038812221578154571664279193151818661085464624362809044421362589886 1
90605651518550768525345880360982620499930023529870923149058909420224779731 2
708961907

n2 =
11784814258076730147914061071035019140036380286211930348614543642438640365 1
94550612460473107888167878530256653589406777969633860665437558791205423666 7
90574516718077954191482160511567555026534864869479185482463499488478409834 7
86061133686325602529264501288347125387119644018588934288470993105694045828 5
831881759

n3 =
10957269009583354001517776304283649384959505726550721493821792863849228527 32
90677719636021241125138870533022624777135621339417905342243382749096555775 7
59988160139635903509463396198660339087469731317110040200424630671076190870 0
69238692928330003893394189130183610367193024615641607787880112240358357912 2
661211123

n4 =
14170137551338516643609685685187037756975719661832330151547456185196394420 1
55389280638563468597971943413462689286751456686967217531805479512706112313 6
34315104638875076885358518496546444777866044124536971729192545681349661601 4
31631588030382487497115804075771206294221501973881473315744538514814957067 0
759906769

n5 =
16084078278888686983255221767903122730614529876024855860423932456464293251 1
80955675872377149302402765235691767263269567784953112776662262120633498101 3
36730523106139644708940373152546558104159776888234216553492427130366890858 2
93623100018638598386938170461346442301774913994022386668443237853704683105 3
915159771
```

```
mods = {
    "n1": n1,
    "n2": n2,
    "n3": n3,
    "n4": n4,
    "n5": n5,
}

keys = list(mods.keys())

for i in range(len(keys)):
    for j in range(i + 1, len(keys)):
        a = keys[i]
        b = keys[j]
        g = gcd(mods[a], mods[b])
        print(f"gcd({a}, {b}) = {g}")
```

Resultado:

```
gcd(n1, n2) = 1
gcd(n1, n3) = 1
gcd(n1, n4) = 1
gcd(n1, n5) = 1
gcd(n2, n3) = 1
gcd(n2, n4) = 1
gcd(n2, n5) = 1
gcd(n3, n4) = 1
gcd(n3, n5) = 1
gcd(n4, n5) = 1
```

Donde para todo par:

- Los módulos son coprimos entre sí.
- No comparten primos.
- **NO podemos factorizar ninguna clave privada por esta vía.**

4.2 Ataque broadcast de Hastad

Recopilamos, tenemos:

- 5 destinatarios criptográficos.
- 5 ciphertexts RSA `encryptedKey`.
- 5 módulos distintos y coprimos.
- 1 exponente público `e = 5` en todos los certificados.

Vamos a resolver: $c_i \equiv m^5 \pmod{n_i}$. Donde:

- c_i es cada `encryptedKey`.

- n_i es el módulo RSA correspondiente.
- `m` es la misma clave de sesión encapsulada para todos.

Como los módulos son coprimos, podremos usar el **Teorema Chino del Resto para reconstruir m^5, y luego calcular la raíz a la quinta para obtener `m`.**

Script python para reconstruir `m` directamente:

```
from math import prod

# Módulos RSA, emparejados por identidad

# Evgeniy Bogachev
n1 =
int("dea580ea11f1dcb7bbf60f288c3f8c74767f447480ff1b97d5b9a235fa151f54f94e86
c0acf4bff7c95473df3ddfa7e8760cb084c0ef706bc8eb1b3a4f9d36b25cb6d399ba1d47bbe
e0e3b2a70c5869969fe83c19d2b41e316dc6c8bb81bba38a2bc9e00d3e29e4bc421293ea0f5
cbea8595573c5731545f57d43742e0f86673", 16)

# Vladimir Levin
n2 =
int("a7d24265db87bf12d397acca8cea0b46a062e414ba0966f019aa598fcdf828708d66e7
a95a50a6742e73e4f5e9094406ba797b031c228d5b754310ce69f09bdcadee50cd053c8da1c
3032259434fbad35d236994ab1489def754635bbb3e61f4ac3e6b2c9a1092ebcbd363446c10
c416d726b05ccb6495c9f3540764c7d5041f", 16)

# Murat Urtembayev
n3 =
int("9c0964865917a807924837a815dba27a9f627005eafdc40c09364c99c66d8ee0d462b0
a0f6ffeedb7a0a07c8d62eefbb0d5f8aa8610b319e23637f6127b088dba29863fe4d275d8b5
c54d1c04ce5a7baf9c285d0c6fb47d2b3f5e7705fc7cf6ddf62ed92842959949015191dc759
5aa4ec9af02b6fc55102f848f0190c3a3bf3", 16)

# Pietr Stepanov
n4 =
int("e50b7d1847bdb28e0e3f9b39884deb6d64a8a0e0eac3e4a27823a6e9f6487122d1ade8
cc9f50b306636ffd5e1c37afb9ee31b3d2bb50da385223630103922aa21a0856f112ad51ea7
32ccac18b1315e6ac19e47e102d01ec64bada7a1a655b7de379fbbbcb4585cb38c534c04257
cbeb3720d1f252162bfc828183bb5615fcdb", 16)

# Gonzaloff Alvarezosky
n5 =
int("c9ca19d889a8f868e5d27043f5b502a078ea6e400fbb0d79da036a28a564ea5e351d6b
04c77faaab2b5c23667f40ec8cc24edadc051995b38613dc1ee0d479bb4e93a1d9cb22febca
38a2fbff91671e0c152569790cdc6d4dd54630cb16976dee1d520fb9316183f43a9bce6889e
4742705aebc78dbf094a296c1bbc84e60dd1", 16)


# encryptedKey, emparejados por identidad

# Evgeniy Bogachev
c1 =
```

```
int("93b2f23d212ad4fe2b8af2cf095accf51d40a49e228a46db124ee48951560755bb0947
e49709fd7c323721b9c75a1a420baafd454e13c4c972a7aa6a6f30924e688ce9b661b104e6c
f2b06cd38eefdd75f1dafa7bf37e41d1ced96b1d6a756cd8dd15e1b481266ebec5ab2285628
25e181513f35d1083d776cc805f748a280de", 16)

# Vladimir Levin
c2 =
int("629a73ab9c2e984540c0e6e51f9b511102188360f1189703e14c6256ae337fedc99586
b270e6caaf806ac834260e3c6acbcd20bb4bc7aaacda740a0e3063afbd475c6deac370303c6
c6fdcc7cc0e4f0ec2032d0ead1a84a7955aa6e0299dd57043f6b056d00796f60f3249b28242
47cca68d70aa8f6e0693af23ad8edd95d627", 16)

# Murat Urtembayev
c3 =
int("0dee7feacc216e79af3e5a33014655912ef8a6b3b95f991e5440117c05e43ab23786cd
3fc8960d4b33308a1e83050552d02160deab7495f2323acf8f8bf0eb917d7ec848c07ca13f7
da00c7d70c7abe53ff58f36d000f62954e029e5deea2bd376317cf67e31dfdd1fe8d8fde422
826bb0e6430c9653d5ce30fc1fbc127bb3ee", 16)

# Pietr Stepanov
c4 =
int("78df06a6e6a828630df0e51e67fc0f4f7095bbe0b03fd1790901c2d167add01e08452a
2e7f5e41c4120c18b0cde3a23c38407baf3452c23cecf6d2b63ed677a92931e435fa9f52d77
62f31125aeefec7317dfc6d3b7124278825a54ef09cd1552a9fc22ee41d77fdf01fdf595db8
08e55fd73ee62c11f83868a3942459276707", 16)

# Gonzaloff Alvarezosky
c5 =
int("b68697d66dd835bac0afd070d47b7c04a31782fffc24b312eac3ee2bb9dd30b096ce73
cdcfd1946689387a5c8a674ceff98ac5d3e961dc1bc1a62d3fa91dbc891a350a7f44f24eab9
da91a7aac4198a18bc76f487c1dc5e4c99bf38434be9c52cdb28b83e587e1d88a991260571d
0923e2c30defa497abd76d3b7a28166a7d80", 16)


pairs = [(c1, n1), (c2, n2), (c3, n3), (c4, n4), (c5, n5)]

def crt(congruences):
    N = prod(n for _, n in congruences)
    x = 0
    for c, n in congruences:
        Ni = N // n
        inv = pow(Ni, -1, n)
        x = (x + c * Ni * inv) % N
    return x, N

def integer_nth_root(x, n):
    lo, hi = 0, 1
    while hi ** n <= x:
        hi *= 2
    while lo + 1 < hi:
        mid = (lo + hi) // 2
        p = mid ** n
        if p == x:
            return mid
```

```
        if p < x:
            lo = mid
        else:
            hi = mid
    return lo

M, N = crt(pairs)
m = integer_nth_root(M, 5)

print("[+] M =", M)
print("[+] m =", m)
print("[+] ¿m^5 == M?", m**5 == M)

if m**5 == M:
    m_bytes = m.to_bytes((m.bit_length() + 7) // 8, "big")
    print("[+] hex(m) =", m_bytes.hex())
    print("[+] len(m_bytes) =", len(m_bytes))
else:
    print("[-] No se obtuvo una quinta potencia exacta.")
```

Obtenemos:

```
python calcular_m.py
[+] M =
41630192139365496122521658004172036592202430478758105774918384698998143749679116407385779005606604268770029815867945280085418643642293103833881138504227561724449183126105316912237291060227499479202507977940830688488788780990444881167214349955569101200715018310488166717353466941665576302417490043343681759202089886878006131711464685057932304025929341888785233721328935873468750000 0
[+] m =
13300920052512658980323276209030077854414546367783598752755980476648996869550
[+] ¿m^5 == M? True
[+] hex(m) =
1d680dd0af91a59b6b3c7bdfff6c9479a85ab8c6c1ac320a0577edcb55f215ae
[+] len(m_bytes) = 32
```

Donde:

- El CRT reconstruyó correctamente $M = m^5$.
- Para obtener `m` aplicamos la raíz a la quinta a `M` y obtenemos: `m = 13300920052512658980323276209030077854414546367783598752755980476648996869550`. Este valor es una clave `AES-256`. Con una longitud de 32 bytes = 256 bits.

Conclusiones

- **Conclusión: El esquema es vulnerable al ataque broadcast de Håstad con `e = 5`.**

- **Hipótesis: Este `m` es muy probablemente la clave simétrica `AES-256` usada para cifrar el contenido del correo, es decir el sobre S/MIME.**
- Hemos roto la capa RSA y hemos recuperado la clave de sesión: `Clave AES-256: 1d680dd0af91a59b6b3c7bdfff6c9479a85ab8c6c1ac320a0577edcb55f215ae`.

5. Obtener la contraseña de EvilHacker.pfx

5.1 Descifrar el encryptedContent con AES-256-CBC

Resumiendo, hasta aquí sabemos que:

- `Clave AES-256`: `1d680dd0af91a59b6b3c7bdfff6c9479a85ab8c6c1ac320a0577edcb55f215ae`.
- `IV - Vector de Inicialización`: `b1a88e22d5583059cc2a1fa1fb596fbb`. El `iv` es un valor inicial de 16 bytes que AES-CBC necesita para empezar el cifrado y descifrado correctamente. No es secreto como la clave, pero sí debe coincidir exactamente.
- `ciphertext`: `f0e8085bf907c96979ab9c6bba4cfef57b78d0e7330063bf2d5480271339f483`.

Primero creamos un fichero con el ciphertext binario:

```
└─$ echo "f0e8085bf907c96979ab9c6bba4cfef57b78d0e7330063bf2d5480271339f483"
| xxd -r -p > ct.bin
```

Segundo, desciframos con openSSL:

```
└─$ openssl enc -aes-256-cbc -d \
-K 1d680dd0af91a59b6b3c7bdfff6c9479a85ab8c6c1ac320a0577edcb55f215ae \
-iv b1a88e22d5583059cc2a1fa1fb596fbb \
-in ct.bin

ImperatorskayaRossiya
```

Donde:

- La clave AES-256 recuperada con Hastad es correcta.
- El `encryptedContent` se ha descifrado correctamente.
- El contenido del mensaje en claro es `ImperatorskayaRossiya`.
- **Hemos descifrado el ciphertext: `ImperatorskayaRossiya`.**

6. Abrir EvilHacker.pfx

Abrimos `EvilHacker.pfx` y escribimos la contraseña anterior:

```
└─$ openssl pkcs12 -in EvilHacker.pfx -nodes -out evilhacker.pem
Enter Import Password:
```

Donde:

- Se comprueba que se pudo extraer el contenido del `PKCS#12` al fichero `evilhacker.pem`.
- Este fichero `evilhacker.pem` incluye el:
    - El certificado.
    - La clave privada

7. Extraer la clave privada de EvilHacker.pfx

Vemos el fichero generado en el paso anterior:

```
└─$ ls
A-M1T1.pdf  bundle.pem  ct.bin  EncryptedFolder.xml  evilhacker.pem
EvilHacker.pfx  mail.msg
```

Vemos el contenido de este fichero `evilhacker.pem`:

```
└─$ cat evilhacker.pem
Bag Attributes
    localKeyID: 37 4C 19 31 15 67 62 AD 07 11 E3 4C DE 23 45 09 D4 B6 F2 F2
subject=C=RU, ST=Moscow, L=Kremlin, O=ChernayaZvezda, CN=Yeguery Burkov
issuer=C=RU, ST=Moscow, L=Kremlin, O=ChernayaZvezda, CN=Yeguery Burkov
-----BEGIN CERTIFICATE-----
MIIDSzCCAjMCFD6xodsW8FFk114YDG5gmuBJU5sLMA0GCSqGSIb3DQEBCwUAMGIx
CzAJBgNVBAYTAlJVMQ8wDQYDVQQIDAZNb3Njb3cxEDAOBgNVBAcMB0tyZW1saW4x
FzAVBgNVBAoMDkNoZXJuYXlhWnZlemRhMRcwFQYDVQQDDA5ZZWd1ZXJ5IEJ1cmtv
djAeFw0yMzA5MjQxNzM5MDdaFw0yMzEwMjQxNzM5MDdaMGIxCzAJBgNVBAYTAlJV
MQ8wDQYDVQQIDAZNb3Njb3cxEDAOBgNVBAcMB0tyZW1saW4xFzAVBgNVBAoMDkNo
ZXJuYXlhWnZlemRhMRcwFQYDVQQDDA5ZZWd1ZXJ5IEJ1cmtvdjCCASIwDQYJKoZI
hvcNAQEBBQADggEPADCCAQoCggEBAL4AnYU1r9VQyHmWiPZPQo4MJQP9QL/5ZUti
Vt+P4pYX2sCXqve4/NIIDkrbZOeW5Sj4XJ2XYSe62Uew4jHfsMAsDqvXdEsQl4lb
iMn2t73nQxZjfQdqCjNi35kBJAC7Dbxa9sdG2x69rOwzCpVJias0ZnaFIv85fGIB
78U3g6sU6RWO45+e5NxPaK/UiYHH6aBOc3LfYvO87u7vIcdF3ao2TQqmLidkvMEC
+b4wNefGDJHzgCN6IZEmeMH16tAPzZ+dV+Pc7zuV0oXmU+kGijVT+PQBn7chYO6G
d1TS4npH1z4iFvTrW4t0HvAfIUo3/O4vCrxcShkKrcyyu5d+iJUCAwEAATANBgkq
hkiG9w0BAQsFAAOCAQEAS2yxo3CI/gH0q4fR2pKKzrg4+7T+/xVjKMky+LXQnG2v
vmTr8Lvs+rvofD39rAbN3AK9N05R3ABKg8UP+BB9lk7vknKhosvNmdvmCiJIRfS1
pbjMVKtAFvlbqt7tcTJi/ULPFF1mGVhNg5IqNIYQFp1bWIPVErwAy3jBa/H64BYM
HQw46+JQptj42g0Aa96h2pO/SoUa5RyxPQuwL+NtlPsiiiE1z9QMMlx4sk5pEr7B
X2KjOv6FjOh1f0GY7r2hzXD/c1w9ZPfG/5+FS/IJDmFSBAcRdp5WVMfRox290h0W
qieNQFxiuRhqRiHnKL3FTLlruUkpDoRggNLWakyEbg==
-----END CERTIFICATE-----
Bag Attributes
    localKeyID: 37 4C 19 31 15 67 62 AD 07 11 E3 4C DE 23 45 09 D4 B6 F2 F2
```

```
Key Attributes: <No Attributes>
-----BEGIN PRIVATE KEY-----
MIIEwAIBADANBgkqhkiG9w0BAQEFAASCBKowggSmAgEAAoIBAQC+AJ2FNa/VUMh5
loj2T0KODCUD/UC/+WVLYlbfj+KWF9rAl6r3uPzSCA5K22TnluUo+Fydl2EnutlH
sOIx37DALA6r13RLEJeJW4jJ9re950MWY30HagozYt+ZASQAuw28WvbHRtsevazs
MwqVSYmrNGZ2hSL/OXxiAe/FN4OrFOkVjuOfnuTcT2iv1ImBx+mgTnNy32LzvO7u
7yHHRd2qNk0Kpi4nZLzBAvm+MDXnxgyR84AjeiGRJnjB9erQD82fnVfj3O87ldKF
5lPpBoo1U/j0AZ+3IWDuhndU0uJ6R9c+Ihb061uLdB7wHyFKN/zuLwq8XEoZCq3M
sruXfoiVAgMBAAECggEBALZ3rM7zAgvf6scp1BqHf6ke7KVqHza98ZN1u2wzqCzw
d7ohbaU9WD70UZV+IOdvuF+iNH97k2guPuBkdLXk1r+XD/fjj+FO69iTuUgL0eVk
JL9ph/cI1QcIWyd7OvIgsU5EJV63I1PgYCLt2aoI11c9mXEU5skokdUlevhxGde/
M0pSh5WzZri2q7WSsoCv1nY8DrhNSahpRqpYvQxFG4acA8/9LPeayP6QGtah+L9z
wUcH7jJDMX622ofiuvd15TAVsUDvU1DqDP59OfjLklX6rwy1ETiSGSIwTOD+mGd1
RXOpP8c3AXgHuUNbOCqvn7flT/ZKCxAJGlBBbLCUZ3kCgYEA1V6AMiCBtWj783Y/
XCYPF5tTq8vKrwzCh6KqkHRGxcnuXtrrgdfRkNBX5fRdYXydiw7qjU040HyAoRDU
nO944/j0hs02gaRTd1AhD0Q3CLdv3ZTlZXODbIqGfAVgBdiUq3r1mm+fYq50KYFL
NAwIB8/6FG5XWTu/sspC6KTJQ+sCgYEA4/bxNFKQEvgsQPC7c7ZR6HjDR3OvGM1A
YmsgVClHk6zvTl3NP+BatEkCgagTibm3PU4OE6cXSMJJqwUJkit137XLiiHuVQ4E
MGbKh5QSE/bOLc7RyNsPjYfYUbswzM8R7obLA+a8B2FJRBgInnbMsVTBhOFV98+3
BRdkw8SmxX8CgYEAkgWJcETE4BUIEJdFR1gYjgxUF8dxX5k47EBLwG6DHMtUYgAq
moLLvfKHpS02VwhEfS1pIejOUAhUs2XkS76rmkD/f92jRtfh806YQW1qyIFZgmhr
EaIbOEYZQXkgPP/6xtxwhw+LIarIfrsob2coe8g+morlJoF4cPBxF+vmfZ8CgYEA
muk5WdvtKebZc7vjNiukF3zQ/uIfD5SS9w5jJq3yhOjUqgVWL90O5kpi2KbVuBaq
7Y74Q+SdDwVwODYxHdJIxJ/uU8zg/42JzTZc1KsLaeOikkBkEMDEHeI9XrT84YCp
Z9TTarBGACHdPteyK7UBPlQhOWJDICZX8a7G+73LspMCgYEA0iCttyA+3q7pew1B
BHKtb1KDCsoVax/ecjn6SrkAQVP9gEFn9oy9jXu9yOqIhz7WBfw+/hsiSibfGWV2
4Xin3mh+AMry/gjwzPrIPEiMRBt1TYXrBdO9Nh239OqPjFHYupi90j35UyFR5WuZ
PHuvseDXgq4rFFtGcye3DWfvo1o=
-----END PRIVATE KEY-----
```

FIN: Usaremos la clave privada de `evilhacker.pem` para atacar `EncryptedFolder.xml`.

8. Analizar y Descifrar EncryptedFolder.xml

Contenido del fichero `EncryptedFolder.xml`:

```
└─$ cat EncryptedFolder.xml
<?xml version="1.0" encoding="UTF-8"?>
<folder encryptionAlg="AES-128-CBC-RSA-2048-PKCS1v15">
<certificate>
-----BEGIN PUBLIC KEY-----
MIIBIjANBgkqhkiG9w0BAQEFAAOCAQ8AMIIBCgKCAQEAvgCdhTWv1VDIeZaI9k9C
jgwlA/1Av/llS2JW34/ilhfawJeq97j80ggOSttk55blKPhcnZdhJ7rZR7DiMd+w
wCwOq9d0SxCXiVuIyfa3vedDFmN9B2oKM2LfmQEkALsNvFr2x0bbHr2s7DMKlUmJ
qzRmdoUi/zl8YgHvxTeDqxTpFY7jn57k3E9or9SJgcfpoE5zct9i87zu7u8hx0Xd
qjZNCqYuJ2S8wQL5vjA158YMkfOAI3ohkSZ4wfXq0A/Nn51X49zvO5XSheZT6QaK
NVP49AGftyFg7oZ3VNLiekfXPiIW9Otbi3Qe8B8hSjf87i8KvFxKGQqtzLK7l36I
lQIDAQAB
-----END PUBLIC KEY-----
</certificate>
```

```
<file name="File01.txt">
<encryptedKey>Hx7aSZvYOJ6GOyPmlfbIrWihtyMZWuPVGRxcASXz5+X7pUoo9lWeDo4xV9izH
YNnhQuAFKTXqxpaPtXKOR5HtMaxvXee0RzMPw9x2ynIox5U+546WyTgcxuXwzjzaNfTLuyW/L1Z
z+k14cJDf2WkcmjJA7N7Wj8J27u7dI8011kJM+kHttFfbyAtap8vev/qRthYYecwaZZDYiM1gKe
Of2W/NmbCG/DawR5HFElxgZvvS0XTRtZ8nBdaOYdMrYBLpUp1Dp1QcrlucDvxWndP4I7uNRA1C8
26/tp8D47ziKuWWhsN+uMwp339pjiG8SS6q6M2KKvGtJUG8OoL2hPfiA==</encryptedKey>
<iv>o16YC4NXZOKx4nHAkkYahQ==</iv>
<encryptedFile>ePXnQqaEZg1veR3PqgRlwVDDjiiIKXwyvmOx6ETlDDbLZ5Ahlr6SPo4gar3f
EddTVmsAywrnd3nrgPmBtKKFaUkZBkTiOM0OODvzEB/PTG0VaGf9gRFAGMeEiMsYD4pcyl8r/P8
OBl2QWcVmF19Z5LrFntNg14xAZTTC54Wl5tkdbXgR7t4QpvWxYqQS18IDdl8JGjIC1FETh+hr1N
8aapTDdMQmuKl4/xRMTKoZ8APoHCJEUUSfmz6mvip8NzotB0OPob1IIuOm4qWKmNscBt1SJvTpA
XgzpQMNYN0kfzZ42LYzZNsdWFEqbsfRAtoE82prUlx4Cbc7CVhTdKlzQux9MwyCzmVKtihveqK6
bIcccOgRXFFaUdeY3DFdw9coUhxzrazjuhUr4eXxgwyhsnD2+5g57weImY3qJOqH84Wfwe18EOB
REkoyBQ3O579GYdVCKvOTznD62+hXFge5BXy1BOE8x8mu2D54fa9FTecOyXNEA0091tDPZdkJq4
+H</encryptedFile>
</file>
<file name="File02.txt">
<encryptedKey>DBUJfOzCYanjo7hjx0e4zLvaVb5Xzm9GpfYK4Gw+bHTnykTEgsm/rUhTjAku2
oemUfRzhazoIY+q01N925vOjJlmjyE8XqCpmsDjOZ/8nDb+XG5pjb7WdYvlgsPWZoCdIWtgCDZ3
lfgInfga0b1yEpLLMzv0n4cU8KPnEr4YJ38AXvJ98xd9TgmhvLCClZFSj6sVod7zn1Bd6Z21dYF
4WfKnH35Meeyfnobqzmcgx Ua/P4GJdXEPmnT/AkzuDGIZIaFsDWBy/y1N6AbDF2W48yxE23WVvt
drltiXMpyN2dgymP4qdEx+941H8bVtAIufzpDo0zUp48AjEpwwVEWeIg==</encryptedKey>
<iv>Ci41mwCuFRuAzEd93qGdtA==</iv>
<encryptedFile>+DHdlK+Qh6OCmGa9LdbsBtRgLsGD6VzGC5Lo/T1fJIFcamWy5rkCbdJbYljx
dG4MZkaynkMhgvZLOYH7OSQwbM2F8DScTNZvaJVtK3/F6HiLvwXfpwLm/EcJO/btftXNMUst3OU
0Bk1v/YVKdznOAJeNYOfLaF98DvBAFceIUggKGZYmRHIBSZu8aJX4VTTTz7RYVwhJfKz6Rfu68E
N7U7GIejV+JoiB0EKq34piviL+9n9he519SL0krp/ILY8jLm95LZRoIMAOGfBTv5yuhqnXO6w1K
P7rrBd1JFpdeHfXljD/e9s9k9IhZnESrdwMTZlBeDTqVgHZ54MepH6ukbCBYKQbsQrvC3IbVlxi
ect570yyvDCWEMgnR7wrj6ZuvDBf3JOEimA1JJ8SRV+VSWhJRbOkNrF4cVyWJUaVGSJiExHHNw1
3+Emkuf7vE55AYOKvCGFAFC+XRr7q+REc6pU8fJ0KbjLtrudp/0rXCHhDdXwBQkpYA4g/SuUInV
0TpXq24CZ6uC7xGizE+061TQJ29q4cKFs/ITL5UJaqdauBaeloTCCc04jDrhYZo4K+WrWvZY+kY
zSWlg1J0e625sOLd1NbIMokd5Vpm7Rwz2k=</encryptedFile>
</file>
<file name="File03.txt">
<encryptedKey>pwR5OoVbtFesT4NFM1NCo605aY0bgXSce0ycsM1KMP54BMoMCz8WAbIYBQzmt
ghkRLgcZjQw9X3uy6PnZVoY5ozCFa4TdjWW3IfXoAWYz2lQgB/NU55YnFZoBbHprocfW7tEA1rp
1L97xWwhQQyBl7gT9rb8w61DDZhtxdARSXA8uc4unJWTsYruHZFdJo5REvB5ui6UbQtmKCLyjMf
HGRkDZBevN4FoQURC2o0myGoeH4Wjzn/6u+sb/Rud3M3t90zqKt3QG9MRl+hI+HpCKUf1iZ16Xq
GoVMLdi8c1AuZGHhPdo2Iia+uQHrJ6B8qDRxlvqAuR8D0xsOp66r/L6g==</encryptedKey>
<iv>UWVdL2tep0/jbh6zTDcSHw==</iv>
<encryptedFile>+tDF9vYO5MK4dy5eWmcgPQBs+pqKzvJoRDJyGfmcaliW1f4szLO/4U6WooUH
O6SGexOeXW5Wvxa86TKkER2xz3CfNzZZrN4488u1UANNvFMeEZPtErQVRP+j3bC78p58zkU2hp+
rH889C5q+9zSLl5GPP0O3AfKsojDqm8FkUXf06mBm/yYmRaHinEMvq8oAdsR3WmndEtVTbBFXa6
eftx7R+rOrxcNntIdqOdFuGqY=</encryptedFile>
</file>
</folder>
```


Donde:

- `encryptedKey` = clave simétrica AES-128 cifrada con RSA-2048 PKCS#1 v1.5.
- `iv` = vector de inicialización de AES-CBC.
- `encryptedFile` = contenido del archivo cifrado con AES-128-CBC.

9. Sacar la clave privada RSA de evilhacker.pem

Extraemos sólo la clave privada del fichero `evilhacker.pem`:

```
└─$ openssl pkey -in evilhacker.pem -out evilhacker_key.pem
```

Mostramos el fichero generado `evilhacker_key.pem` que contiene sólo la clave privada:

```
└─$ cat evilhacker_key.pem
-----BEGIN PRIVATE KEY-----
MIIEwAIBADANBgkqhkiG9w0BAQEFAASCBKowggSmAgEAAoIBAQC+AJ2FNa/VUMh5
loj2T0KODCUD/UC/+WVLYlbfj+KWF9rAl6r3uPzSCA5K22TnluUo+Fydl2EnutlH
sOIx37DALA6r13RLEJeJW4jJ9re950MWY30HagozYt+ZASQAuw28WvbHRtsevazs
MwqVSYmrNGZ2hSL/OXxiAe/FN4OrFOkVjuOfnuTcT2iv1ImBx+mgTnNy32LzvO7u
7yHHRd2qNk0Kpi4nZLzBAvm+MDXnxgyR84AjeiGRJnjB9erQD82fnVfj3O87ldKF
5lPpBoo1U/j0AZ+3IWDuhndU0uJ6R9c+Ihb061uLdB7wHyFKN/zuLwq8XEoZCq3M
sruXfoiVAgMBAAECggEBALZ3rM7zAgvf6scp1BqHf6ke7KVqHza98ZN1u2wzqCzw
d7ohbaU9WD70UZV+IOdvuF+iNH97k2guPuBkdLXk1r+XD/fjj+FO69iTuUgL0eVk
JL9ph/cI1QcIWyd7OvIgsU5EJV63I1PgYCLt2aoI11c9mXEU5skokdUlevhxGde/
M0pSh5WzZri2q7WSsoCv1nY8DrhNSahpRqpYvQxFG4acA8/9LPeayP6QGtah+L9z
wUcH7jJDMX622ofiuvd15TAVsUDvU1DqDP59OfjLklX6rwy1ETiSGSIwTOD+mGd1
RXOpP8c3AXgHuUNbOCqvn7flT/ZKCxAJGlBBbLCUZ3kCgYEA1V6AMiCBtWj783Y/
XCYPF5tTq8vKrwzCh6KqkHRGxcnuXtrrgdfRkNBX5fRdYXydiw7qjU040HyAoRDU
nO944/j0hs02gaRTd1AhD0Q3CLdv3ZTlZXODbIqGfAVgBdiUq3r1mm+fYq50KYFL
NAwIB8/6FG5XWTu/sspC6KTJQ+sCgYEA4/bxNFKQEvgsQPC7c7ZR6HjDR3OvGM1A
YmsgVClHk6zvTl3NP+BatEkCgagTibm3PU4OE6cXSMJJqwUJkit137XLiiHuVQ4E
MGbKh5QSE/bOLc7RyNsPjYfYUbswzM8R7obLA+a8B2FJRBgInnbMsVTBhOFV98+3
BRdkw8SmxX8CgYEAkgWJcETE4BUIEJdFR1gYjgxUF8dxX5k47EBLwG6DHMtUYgAq
moLLvfKHpS02VwhEfS1pIejOUAhUs2XkS76rmkD/f92jRtfh806YQW1qyIFZgmhr
EaIbOEYZQXkgPP/6xtxwhw+LIarIfrsob2coe8g+morlJoF4cPBxF+vmfZ8CgYEA
muk5WdvtKebZc7vjNiukF3zQ/uIfD5SS9w5jJq3yhOjUqgVWL90O5kpi2KbVuBaq
7Y74Q+SdDwVwODYxHdJIxJ/uU8zg/42JzTZc1KsLaeOikkBkEMDEHeI9XrT84YCp
Z9TTarBGACHdPteyK7UBPlQhOWJDICZX8a7G+73LspMCgYEA0iCttyA+3q7pew1B
BHKtb1KDCsoVax/ecjn6SrkAQVP9gEFn9oy9jXu9yOqIhz7WBfw+/hsiSibfGWV2
4Xin3mh+AMry/gjwzPrIPEiMRBt1TYXrBdO9Nh239OqPjFHYupi90j35UyFR5WuZ
PHuvseDXgq4rFFtGcye3DWfvo1o=
-----END PRIVATE KEY-----
```

A continuación, la idea es, para cada `file`:

- Sacar `encryptedKey`,
- decodificarlo desde `base64`,
- descifrarlo con la clave privada RSA,
- obtener la clave `AES-128`,
- decodificar `iv`,
- decodificar `encryptedFile`,
- y descifrar con `AES-128-CBC`.

10. FILE 1

Analizando el fichero `EncryptedFolder.xml` sabemos que para File 1 el valor de `encryptedKey` es `Hx7aSZvYOJ6GOyPmlfbIrWihtyMZWuPVGRxcASXz5+X7pUoo9lWeDo4xV9izHYNnhQuAFKTXqxpaPtXKOR5HtMaxvXee0RzMPw9x2ynIox5U+546WyTgcxuXwzjzaNfTLuyW/L1Zz+k14cJDf2WkcmjJA7N7Wj8J27u7dI8011kJM+kHttFfbyAtap8vev/qRthYYecwaZZDYiM1gKeOf2W/NmbCG/DawR5HFElxgZvvS0XTRtZ8nBdaOYdMrYBLpUp1Dp1QcrlucDvxWndP4I7uNRA1C826/tp8D47ziKuWWhsN+uMwp339pjiG8SS6q6M2KKvGtJUG8OoL2hPfiA==`.

Extraer y decodificar encryptedKey:

```
└─$ echo
'Hx7aSZvYOJ6GOyPmlfbIrWihtyMZWuPVGRxcASXz5+X7pUoo9lWeDo4xV9izHYNnhQuAFKTXqx
paPtXKOR5HtMaxvXee0RzMPw9x2ynIox5U+546WyTgcxuXwzjzaNfTLuyW/L1Zz+k14cJDf2Wkc
mjJA7N7Wj8J27u7dI8011kJM+kHttFfbyAtap8vev/qRthYYecwaZZDYiM1gKeOf2W/NmbCG/Da
wR5HFElxgZvvS0XTRtZ8nBdaOYdMrYBLpUp1Dp1QcrlucDvxWndP4I7uNRA1C826/tp8D47ziKu
WWhsN+uMwp339pjiG8SS6q6M2KKvGtJUG8OoL2hPfiA==' | base64 -d > key1.enc
```

Descifrar encryptedKey con la clave privada RSA: Usamos la clave privada extrida anteriormente:

```
└─$ openssl pkeyutl -decrypt -inkey evilhacker_key.pem -in key1.enc -out
key1.bin
```

Decodificar el iv:

```
└─$ echo 'o16YC4NXZOKx4nHAkkYahQ==' | base64 -d > iv1.bin
```

Decodificar encryptedFile: Para `File01.txt`, el `encryptedFile` es el bloque largo en `base64`. Guardamos este bloque en el fichero `file1.enc`:

```
└─$ echo
'ePXnQqaEZg1veR3PqgRlwVDDjiiIKXwyvmOx6ETlDDbLZ5Ahlr6SPo4gar3fEddTVmsAywrnd3
nrgPmBtKKFaUkZBkTiOM0OODvzEB/PTG0VaGf9gRFAGMeEiMsYD4pcyl8r/P8OBl2QWcVmF19Z5
LrFntNg14xAZTTC54Wl5tkdbXgR7t4QpvWxYqQS18IDdl8JGjIC1FETh+hr1N8aapTDdMQmuKl4
/xRMTKoZ8APoHCJEUUSfmz6mvip8NzotB0OPob1IIuOm4qWKmNscBt1SJvTpAXgzpQMNYN0kfzZ
42LYzZNsdWFEqbsfRAtoE82prUlx4Cbc7CVhTdKlzQux9MwyCzmVKtihveqK6bIcccOgRXFFaUd
eY3DFdw9coUhxzrazjuhUr4eXxgwyhsnD2+5g57weImY3qJOqH84Wfwe18EOBREkoyBQ3O579GY
dVCKvOTznD62+hXFge5BXy1BOE8x8mu2D54fa9FTecOyXNEA0091tDPZdkJq4+H' | base64 -
d > file1.enc
```

Descifrar encryptedFile con AES-128-CBC: Usaremos:

- Clave = contenido de `key1.bin`.
- `IV` = contenido de `iv1.bin`.

```
└─$ openssl enc -aes-128-cbc -d \
-in file1.enc \
-out File01.txt \
-K $(xxd -p key1.bin | tr -d '\n') \
-iv $(xxd -p iv1.bin | tr -d '\n')
```

Vemos el fichero generado `File01.txt`:

```
└─$ cat File01.txt
Ed Dillindzher, starshiy

Biograficheskaya informatsiya
Imya pol'zovatelya 00 — Dillindzher
Status zhiv

Fizicheskoye opisaniye
Muzhskoy pol
Tsvet Volos Blond
Tsvet glaz siniy

Dopolnitel'naya informatsiya
Funktsii Starshiy ispolnitel'nyy direktor ENCOM (byvshiy programmist ENCOM)
Programmy MKP
Sark
Vertolet
Soyuzniki Pitera
Edvard Dillindzher-mladshiy
MKP
```

donde:

- **En `File01.txt` no contiene credenciales bancarias.**

11. FILE 2

Repetimos todo el proceso para File 2 y obtenemos:

```
└─$ echo
'DBUJfOzCYanjo7hjx0e4zLvaVb5Xzm9GpfYK4Gw+bHTnykTEgsm/rUhTjAku2oemUfRzhazoIY
+q01N925vOjJlmjyE8XqCpmsDjOZ/8nDb+XG5pjb7WdYvlgsPWZoCdIWtgCDZ3lfgInfga0b1yE
pLLMzv0n4cU8KPnEr4YJ38AXvJ98xd9TgmhvLCClZFSj6sVod7zn1Bd6Z21dYF4WfKnH35Meeyf
nobqzMcgxUa/P4GJdXEPmnT/AkzuDGIZIaFsDWBy/y1N6AbDF2W48yxE23WVvtdrltiXMpyN2dg
ymP4qdEx+941H8bVtAIufzpDo0zUp48AjEpwwVEWeIg==' | base64 -d > key2.enc
```

```
└─$ openssl pkeyutl -decrypt -inkey evilhacker_key.pem -in key2.enc -out
key2.bin


└─$ echo 'Ci41mwCuFRuAzEd93qGdtA==' | base64 -d > iv2.bin


└─$ echo
'+DHdlK+Qh6OCmGa9LdbsBtRgLsGD6VzGC5Lo/T1fJIFcamWy5rkCbdJbYljxdG4MZkaynkMhgv
ZLOYH7OSQwbM2F8DScTNZvaJVtK3/F6HiLvwXfpwLm/EcJO/btftXNMUst3OU0Bk1v/YVKdznOA
JeNYOfLaF98DvBAFceIUggKGZYmRHIBSZu8aJX4VTTTz7RYVwhJfKz6Rfu68EN7U7GIejV+JoiB
0EKq34piviL+9n9he519SL0krp/ILY8jLm95LZRoIMAOGfBTv5yuhqnXO6w1KP7rrBd1JFpdeHf
XljD/e9s9k9IhZnESrdwMTZlBeDTqVgHZ54MepH6ukbCBYKQbsQrvC3IbVlxiect570yyvDCWEM
gnR7wrj6ZuvDBf3JOEimA1JJ8SRV+VSWhJRbOkNrF4cVyWJUaVGSJiExHHNw13+Emkuf7vE55AY
OKvCGFAFC+XRr7q+REc6pU8fJ0KbjLtrudp/0rXCHhDdXwBQkpYA4g/SuUInV0TpXq24CZ6uC7x
GizE+061TQJ29q4cKFs/ITL5UJaqdauBaeloTCCc04jDrhYZo4K+WrWvZY+kYzSWlg1J0e625sO
Ld1NbIMokd5Vpm7Rwz2k=' | base64 -d > file2.enc


└─$ openssl enc -aes-128-cbc -d \
-in file2.enc \
-out File02.txt \
-K $(xxd -p key2.bin | tr -d '\n') \
-iv $(xxd -p iv2.bin | tr -d '\n')


└─$ cat File02.txt
Sem Flinn

Biograficheskaya informatsiya
Imya pol'zovatelya
Psevdonimy Syn Flinna
Data rozhdeniya 1983 g.
Status zhiv

Fizicheskoye opisaniye
Muzhskoy pol
Tsvet volos zolotisto-korichnevyy

Dopolnitel'naya informatsiya
Funktsii Iskatel' priklyucheniy
General'nyy direktor ENKOM
Identifikatsionnyy disk oborudovaniya
Baton
Transportnyye sredstva Light Cycle (5-ye pokoleniye)

Soyuzniki Kevin Flinn
Dzhordan Kanas
Alan Bredli
Kvorra
```

donde:

- `File02.txt no` **tampoco contiene credenciales bancarias.**

12. FILE 3

Repetimos el proceso para File 3:

```
└─$ echo
'pwR5OoVbtFesT4NFM1NCo605aY0bgXSce0ycsM1KMP54BMoMCz8WAbIYBQzmtghkRLgcZjQw9X
3uy6PnZVoY5ozCFa4TdjWW3IfXoAWYz2lQgB/NU55YnFZoBbHprocfW7tEA1rp1L97xWwhQQyBl
7gT9rb8w61DDZhtxdARSXA8uc4unJWTsYruHZFdJo5REvB5ui6UbQtmKCLyjMfHGRkDZBevN4Fo
QURC2o0myGoeH4Wjzn/6u+sb/Rud3M3t90zqKt3QG9MRl+hI+HpCKUf1iZ16XqGoVMLdi8c1AuZ
GHhPdo2Iia+uQHrJ6B8qDRxlvqAuR8D0xsOp66r/L6g==' | base64 -d > key3.enc


└─$ openssl pkeyutl -decrypt -inkey evilhacker_key.pem -in key3.enc -out
key3.bin


└─$ echo 'UWVdL2tep0/jbh6zTDcSHw==' | base64 -d > iv3.bin


└─$ echo
'+tDF9vYO5MK4dy5eWmcgPQBs+pqKzvJoRDJyGfmcaliW1f4szLO/4U6WooUHO6SGexOeXW5Wvx
a86TKkER2xz3CfNzZZrN4488u1UANNvFMeEZPtErQVRP+j3bC78p58zkU2hp+rH889C5q+9zSLl
5GPP0O3AfKsojDqm8FkUXf06mBm/yYmRaHinEMvq8oAdsR3WmndEtVTbBFXa6eftx7R+rOrxcNn
tIdqOdFuGqY=' | base64 -d > file3.enc


└─$ openssl enc -aes-128-cbc -d \
-in file3.enc \
-out File03.txt \
-K $(xxd -p key3.bin | tr -d '\n') \
-iv $(xxd -p iv3.bin | tr -d '\n')


└─$ cat File03.txt
Sberbank
login: gonzalo
pwd: 6S7Sff_4HP+yZntA

Gazprombank
login: gonzalo.alvarez
pwd: MH6KNu;D'm4p/"#}=X+9k

Promsvyazbank
login: 4087 8907 1200 5611
pwd: 11002288
```

donde:

- **En File 3 sí aparecen las credenciales bancarias.**
  - En Sberbank aparece:
    - Un login,
    - y una contraseña.
  - En Gazprombank aparece:
    - Un login,
    - y una contraseña.
  - En Promsvyazbank aparece:
    - Un identificador de acceso,
    - y una contraseña

13. Las credenciales de Gazprombank

Las credenciales bancarias de Gazprombank son:

```
Gazprombank
login: gonzalo.alvarez
pwd: MH6KNu;D'm4p/"#}=X+9k
```

14. Conclusiones

En esta tarea hemos conseguido hacer una ruptura de un esquema de cifrado híbrido basado en RSA y AES, siguiendo una metodología de criptoanálisis explicada en la tería del módulo.

En primer lugar, se analizaron los certificados contenidos en `bundle.pem`, comprobando que se trataba de cinco certificados RSA de 1024 bits, todos ellos con exponente público bajo `e = 5`. Posteriormente, se examinó el fichero `mail.msg`, identificando que contenía un objeto `S/MIME` de tipo `PKCS#7/CMS EnvelopedData`, codificado en `base64`, con cinco destinatarios criptográficos distintos.

A continuación, se extrajeron los cinco módulos RSA y los cinco valores `encryptedKey` asociados a cada destinatario. Tras verificar que no existían factores comunes entre los módulos mediante el cálculo de los correspondientes máximos comunes divisores, se descartó la factorización por primos compartidos como vector de ataque. Esto permitió concluir que la vía correcta era un ataque `broadcast de Håstad`, aprovechando la existencia de cinco destinatarios y el uso del exponente público pequeño `e = 5`.

Aplicando el `Teorema Chino del Resto` y calculando posteriormente la raíz quinta entera, se reconstruyó con éxito el mensaje `RSA original`, obteniéndose la clave simétrica empleada para proteger el contenido del correo. Con dicha clave y el vector de inicialización recuperado del contenedor CMS, se descifró el contenido del mensaje, obteniendo la contraseña necesaria para abrir el fichero `EvilHacker.pfx`.

Una vez desbloqueado el contenedor `PKCS#12`, se extrajo la clave privada correspondiente y se utilizó para descifrar las claves simétricas contenidas en `EncryptedFolder.xml`. Con ello fue posible recuperar los tres archivos cifrados del contenedor XML. El análisis de su contenido mostró que los dos primeros archivos contenían información descriptiva no relevante, mientras que el tercero incluía las entradas sensibles buscadas, entre ellas la correspondiente a `Gazprombank`.